BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO
ĐỒ ÁN MÔN HỌC

ĐỀ TÀI: Xây dựng bộ dữ liệu về bất động sản, thực hiện các phương pháp tối ưu hiệu năng cho hệ thống khi dữ liệu quá lớn

Môn học: Nhập môn Khoa học dữ liệu

Giảng viên hướng dẫn: Gv Nguyễn Ngọc Duy

Thực hiện bởi nhóm sinh viên, bao gồm:

1.	Nguyễn Minh Châu	N23DCCN143	D23CQCNHT01-N
2.	Bùi Xuân Phú	N23DCCN182	D23CQCNHT01-N
3.	Nguyễn Quốc Thái	N23DCCN191	D23CQCNHT01-N

LỜI MỞ ĐẦU

Dữ liệu bất động sản là một ví dụ tiêu biểu của dữ liệu vận hành có tính đa dạng cao: một tin đăng vừa chứa văn bản, địa danh, giá, diện tích và dấu thời gian, vừa có khả năng thiếu một phần thông tin. Trong môi trường dữ liệu lớn, giá trị của một tập dữ liệu không chỉ được xác định bởi số dòng mà còn bởi khả năng kiểm tra quy tắc làm sạch, truy vết nguồn, tái tạo kết quả và khai thác hiệu quả sau khi lưu trữ.

Báo cáo này trình bày một công trình Data Science theo hướng engineering và thực nghiệm cơ sở dữ liệu. Trọng tâm là xây dựng pipeline từ nguồn streaming, chuẩn hóa các trường có liên quan, tạo CSV nhất quán, nạp PostgreSQL an toàn và chuẩn bị benchmark. Do project không có mô hình dự đoán hoặc nhãn mục tiêu, báo cáo không mô tả đây là bài toán Machine Learning; điều đó giúp phạm vi khoa học của đề tài rõ ràng và tránh các kết luận không có trong source code.

Số liệu EDA và benchmark trong báo cáo được truy vết về các artifact chính thức của lượt build 50K, 200K và 500K; mỗi bộ có manifest và mã băm SHA-256. Cả ba tập dữ liệu 50K, 200K và 500K hiện có đã được đối chiếu SHA-256 với manifest chính thức, xác nhận tính toàn vẹn của các tệp sử dụng trong thực nghiệm. Các artifact và thông tin môi trường được lưu trữ nhằm hỗ trợ kiểm chứng kết quả. Phần PostgreSQL sử dụng gói thực nghiệm đã lưu 600 execution plan (3 quy mô × 10 cấu hình × 20 lượt đo), có môi trường, catalog và kiểm tra tính toàn vẹn kèm theo.

TÓM TẮT

Đồ án sử dụng nguồn Phuscca/vietnam-real-estates được đọc bằng chế độ streaming và tạo ba tập CSV chuẩn 9 cột gồm 50.000, 200.000 và 500.000 bản ghi. Bản 50K chính thức có 3.993 giá thiếu, 1.507 quận/huyện thiếu và 7.277 phường/xã thiếu; các giá trị thiếu được bảo toàn dưới dạng NULL thay vì được nội suy.

Pipeline sau làm sạch được thiết kế để nạp vào PostgreSQL bằng COPY trong một transaction. Trước khi commit, importer kiểm tra header, schema, số bản ghi, tính duy nhất/liên tục của ID và số NULL. Bốn truy vấn benchmark mô phỏng các tình huống lọc theo tỉnh, lọc đa điều kiện có sắp xếp, lọc theo thời gian và tổng hợp nhóm. Các index B-tree được gắn với từng kịch bản và được reset độc lập nhằm giảm nhiễu giữa các lần đo.

Gói nghiệm thu PostgreSQL lưu 600 plan thực tế, với ba lượt warm-up và 20 lượt đo chính thức cho mỗi cấu hình. Kết quả regression toàn diện của hệ thống đạt 220/220 passed (100% Đạt); đây là kết quả kiểm thử tự động toàn vẹn trên môi trường thực nghiệm. Báo cáo sử dụng median, IQR, plan và kích thước index để diễn giải kết quả, thay vì dựa vào một lượt đo đơn lẻ.

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU 8

1.1 Vấn đề nghiên cứu và câu hỏi đặt ra 8

1.2 Mục tiêu, phạm vi và giới hạn 9

1.3 Phương pháp nghiên cứu và cấu trúc báo cáo 9

CHƯƠNG 2 CƠ SỞ KHOA HỌC VÀ CÔNG NGHỆ 11

2.1 Chất lượng dữ liệu và tiền xử lý 11

2.2 Xử lý theo luồng và tái lập dữ liệu 11

2.3 Hệ quản trị cơ sở dữ liệu và tính nguyên tử 12

2.4 Chỉ mục B-tree và tối ưu truy vấn 12

2.5 Thiết kế thí nghiệm và chỉ số đánh giá 13

CHƯƠNG 3 KHẢO SÁT VÀ TIỀN XỬ LÝ DỮ LIỆU 14

3.1 Nguồn dữ liệu và lược đồ chuẩn hóa 14

3.1.1 Phạm vi dữ liệu được sử dụng trong báo cáo 15

3.2 Kết quả build và làm sạch 16

3.2.1 Đọc đúng ý nghĩa các bộ đếm trong build report 16

3.3 Dữ liệu thiếu sau chuẩn hóa 17

3.3.1 Mẫu thiếu theo địa lý và ý nghĩa phân tích 18

3.4 Phân bố theo tỉnh thành 19

3.4.1 Mức độ tập trung không gian của tập tin đăng 20

3.5 Loại bất động sản và truy vấn tổng hợp 21

3.5.1 Quy mô từng nhóm và khả năng diễn giải Q4 21

3.6 Phân phối giá niêm yết 22

3.6.1 Phân vị, đuôi phân phối và so sánh theo loại 23

3.7 Phân phối diện tích và giá trị ngoại biên 25

3.7.1 Đặc trưng diện tích theo mức phân vị và loại 25

3.8 Quan hệ mô tả giữa diện tích và giá 27

3.8.1 Cách đọc tương quan mà không suy diễn nhân quả 27

3.9 Dấu thời gian và tính sẵn sàng cho Q3 28

3.9.1 Mật độ dữ liệu thời gian và kiểm tra ngữ nghĩa query 29
CHƯƠNG 4 THIẾT KẾ PIPELINE VÀ CƠ SỞ DỮ LIỆU 30
4.1 Hiện thực làm sạch dùng chung 30
4.1.1 Trình tự chuyển đổi dữ liệu trong clean_dataframe 31
4.2 Xây dựng dataset lớn và công bố an toàn 31
4.2.1 Giới hạn bộ nhớ, file tạm và tính toàn vẹn publish 32
4.3 Schema PostgreSQL và ràng buộc 33
4.3.1 Ánh xạ từ schema CSV sang schema quan hệ 34
4.4 Quy trình import và đối chiếu trước commit 35
4.4.1 Từ kiểm tra file đến transaction đã xác nhận 35
4.5 Query nghiệm thu và chiến lược index 36
4.5.1 Phân tích từng kịch bản SQL theo predicate và thứ tự cột 36
4.6 Tổ chức kiểm thử và khả năng bảo trì 37
4.6.1 Giá trị và ranh giới của kiểm thử offline 38
CHƯƠNG 5 THỰC NGHIỆM VÀ ĐÁNH GIÁ 39
5.1 Môi trường, dữ liệu và giao thức đo 39
5.1.1 Kiểm toán đầu vào và ranh giới tái lập 40
5.2 Kết quả đầy đủ theo 20 lượt đo 40
5.3 Q1 và Q2: index theo điều kiện lọc, thứ tự và giới hạn 42
5.4 Q3: tác động kết hợp của query rewriting và index 43
5.5 Q4: đối chứng âm và chi phí lưu trữ 44
5.6 Kiểm thử, hạn chế và khả năng tái lập 47
5.7 Kết luận thực nghiệm 48
CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 50
6.1 Kết quả đạt được và hạn chế 50
6.2 Hướng phát triển 50
TÀI LIỆU THAM KHẢO 52

DANH MỤC HÌNH

Hình 1: Mức độ thiếu của các trường quan trọng trong CSV 50.000 dòng 17

Hình 2: Tỷ lệ thiếu giá tại mười tỉnh thành có nhiều tin nhất 18
Hình 3: Mười tỉnh thành có nhiều tin đăng nhất 19
Hình 4: Phân bố loại bất động sản theo nhãn nguồn 21
Hình 5: Phân phối log10 của giá niêm yết 22
Hình 6: Giá trung vị theo loại bất động sản; đơn vị tỷ VND 24
Hình 7: Phân phối diện tích trong tập dữ liệu đã làm sạch 25
Hình 8: Diện tích trung vị theo loại bất động sản 26
Hình 9: Quan hệ mô tả giữa diện tích và giá niêm yết 27
Hình 10: Số tin đăng theo tháng 28
Hình 11: Pipeline dữ liệu và đánh giá hiệu năng của project 30
Hình 12: Chuỗi kiểm soát khi nạp CSV vào PostgreSQL 33
Hình 13: Độ trễ trung vị theo quy mô dữ liệu trong benchmark 42
Hình 14: Speedup theo quy mô của các cấu hình benchmark 43
Hình 15: Phân rã nhân tố Q3: query rewriting và index 44
Hình 16: Đối chứng Q4: index không cải thiện aggregate toàn bảng 45
Hình 17: Kích thước index bổ sung theo quy mô dữ liệu 46

DANH MỤC BẢNG

Bảng 1: Mục tiêu và phạm vi nghiên cứu 9
Bảng 2: Công nghệ và vai trò 13
Bảng 3: Lược đồ tập dữ liệu chuẩn hóa 14
Bảng 4: Hợp đồng dữ liệu giữa CSV, pipeline và PostgreSQL 15
Bảng 5: Diễn giải các chỉ số của lượt build chung ba quy mô 50K, 200K và 500K 17
Bảng 6: Mười tỉnh thành có nhiều tin nhất trong CSV 50K 20
Bảng 7: Quy mô nhóm property_type_name 22
Bảng 8: Số tin theo tháng đăng 29
Bảng 9: Ràng buộc schema PostgreSQL 33
Bảng 10: Kiểm tra song song giữa pipeline và database 34
Bảng 11: Ma trận query và index 36
Bảng 12: Cấu hình benchmark và câu hỏi cần trả lời 37
Bảng 13: Môi trường và giao thức benchmark PostgreSQL 39
Bảng 14: Kiểm toán ba dataset chính thức 40
Bảng 15: Kết quả median và IQR của 30 cấu hình 40
Bảng 16: Kết quả Q1 và Q2 kèm kế hoạch quét 43
Bảng 17: Ma trận thời gian thực thi Q3 theo hai yếu tố rewrite và index (ms) 44
Bảng 18: Q4 và chi phí index bổ sung 45
Bảng 19: Trạng thái kiểm thử và khả năng tái lập 48

CHƯƠNG 1: GIỚI THIỆU

Trong các hệ thống đăng tin, dữ liệu tăng nhanh hơn khả năng kiểm tra thủ công. Một truy vấn lọc theo tỉnh có thể chấp nhận được với vài nghìn dòng, nhưng khi số tin tăng, việc quét tuần tự và sắp xếp nhiều bản ghi có thể làm chậm thời gian phản hồi. Song song với hiệu năng, dữ liệu thiếu hoặc định dạng không thống nhất có thể dẫn đến kết quả lọc, tổng hợp và đánh giá sai.

Đề tài được đặt trong bối cảnh cần nối liền ba lớp công việc: dữ liệu đầu vào phải có quy tắc chất lượng minh bạch; dữ liệu đầu ra phải có schema bền vững và có thể nạp; thí nghiệm truy vấn phải không nhầm lẫn giữa sự tồn tại của index và hiệu quả thực tế của index. Ba lớp này tạo thành một quy trình Data Science hoàn chỉnh dù không có bước huấn luyện mô hình.

Mục tiêu của chương mở đầu là xác định đúng bản chất bài toán, ranh giới dữ liệu và tiêu chuẩn đánh giá. Những tiêu chuẩn này chi phối cách diễn giải tất cả số liệu ở các chương sau.

1.1 Vấn đề nghiên cứu và câu hỏi đặt ra

Câu hỏi thứ nhất là làm thế nào để xây dựng tập dữ liệu bất động sản có cấu trúc chuẩn khi nguồn có thể chứa chuỗi rỗng, giá không hợp lệ, địa chỉ thiếu hoặc ngày không đọc được. Câu hỏi thứ hai là làm sao nạp dữ liệu mà không để lại bảng ở trạng thái nửa chừng. Câu hỏi thứ ba là làm sao thiết kế phép so sánh query/index không bị ảnh hưởng bởi chỉ mục còn sót lại hoặc số đo đơn lẻ.

Các câu hỏi được giải bằng source code hiện có, không dựa trên giả định về mô hình dự báo. Module cleaning.py xác định ràng buộc; build_large_dataset.py xử lý nguồn theo batch; import_csv.py quản trị transaction; thư mục sql/benchmark quy định truy vấn, index và reset. Việc tách các nhiệm vụ như vậy làm cho từng lớp có thể kiểm thử riêng.

Bài toán được đánh giá bằng các đối tượng có thể quan sát: số dòng trước/sau lọc, số NULL, schema, ID, query text, index definition, kết quả unit test và sau này là JSON EXPLAIN. Không sử dụng accuracy, F1, RMSE hoặc các metric ML vì chúng không liên quan đến mục tiêu hiện thực của project.

1.2 Mục tiêu, phạm vi và giới hạn

Bảng 1: Mục tiêu và phạm vi nghiên cứu

Mục tiêu	Cơ chế hiện thực	Minh chứng
Chuẩn hóa dữ liệu	Hàm clean_dataframe dùng chung	Code và unit test
Tạo dataset quy mô	Streaming, batch, file tạm	Build report
Import an toàn	COPY giao dịch và đối chiếu	Code và unit test
Benchmark thực tế	3 warm-up + 20 lượt/cấu hình; EXPLAIN ANALYZE JSON	600 plan có manifest, median và IQR

Mục tiêu tổng quát là xây dựng dữ liệu và kịch bản đánh giá truy vấn có thể tái lập. Các mục tiêu cụ thể gồm chuẩn hóa dữ liệu đầu vào, bảo toàn giá trị thiếu có ý nghĩa, tạo target kích thước khác nhau từ một stream, bảo vệ import bằng transaction và thiết kế benchmark có số lần lặp rõ ràng.

Phạm vi dữ liệu là ba bộ chính thức properties_50000.csv, properties_200000.csv và properties_500000.csv được tạo trong cùng một lượt build. Cả ba tập dữ liệu (50K, 200K, 500K) trên workspace đều được xác thực mã băm SHA-256 trùng khớp hoàn toàn với manifest nghiệm thu; các thống kê chính thức trong báo cáo được đối chiếu đồng bộ và nhất quán giữa manifest, quality report và evidence thực nghiệm.

Benchmark được thực hiện trên môi trường Windows với PostgreSQL 18.3. Các execution plan, thông số môi trường và kết quả thực nghiệm được lưu trong gói evidence. Cụm PostgreSQL nội bộ trên cổng 5433 được ghi nhận phục vụ kiểm tra kết nối, nạp dữ liệu và đối chứng kết quả. Việc xác minh tính toàn vẹn của gói benchmark được phân biệt với việc thực thi lại toàn bộ phép đo.

1.3 Phương pháp nghiên cứu và cấu trúc báo cáo

Phương pháp nghiên cứu của đồ án bao gồm: xây dựng quy trình tiền xử lý và pipeline streaming dữ liệu, phân tích khám phá dữ liệu (EDA), thiết kế schema và các chỉ mục tối ưu trên PostgreSQL, và thực hiện đo lường hiệu năng bằng các kịch bản SQL thực nghiệm lặp lại có đối chứng. Cách tiếp cận này bảo đảm tính nhất quán chặt chẽ giữa thiết kế thuật toán, mã nguồn thực thi và các số liệu đo lường thu được.

Báo cáo được tổ chức theo tiến trình từ xây dựng dữ liệu đến đánh giá hiệu năng hệ thống. Chương 2 trình bày cơ sở khoa học và công nghệ; Chương 3 khảo sát nguồn dữ liệu, quy trình tiền xử lý và kết quả phân tích khám phá; Chương 4 mô tả thiết kế pipeline, lược đồ PostgreSQL, quy trình nạp dữ liệu và chiến lược tối ưu truy vấn;

Chương 5 trình bày giao thức thực nghiệm, kết quả đo lường và phân tích hiệu năng; Chương 6 tổng kết kết quả đạt được, các giới hạn và hướng phát triển.

CHƯƠNG 2 CƠ SỞ KHOA HỌC VÀ CÔNG NGHỆ

Chương này chỉ giới thiệu các khái niệm trực tiếp được dùng trong pipeline. Mục tiêu không phải là liệt kê mọi thuật toán của Data Science, mà là làm rõ tại sao các quy tắc làm sạch, xử lý streaming, transaction, index và benchmark có ý nghĩa khoa học và kỹ thuật.

Theo cách nhìn Data Science, dữ liệu chỉ đáng tin cậy khi quy trình biến đổi được mô tả, có thể chạy lại và có tiêu chí kiểm tra. Điều này đặc biệt cần thiết với nguồn mở, vì dữ liệu upstream có thể thay đổi hoặc chứa nhiều biểu diễn khác nhau của cùng một giá trị. [1], [2]

Các phần bên dưới liên hệ khái niệm với những file cụ thể của project để tránh lý thuyết tách rời hiện thực.

2.1 Chất lượng dữ liệu và tiền xử lý

Tiền xử lý dữ liệu là quá trình biến dữ liệu nguồn thành dạng có thể sử dụng mà vẫn bảo toàn ý nghĩa cần thiết. Một lỗi phổ biến là đồng nhất dữ liệu thiếu với dữ liệu sai hoặc coi mọi ngoại lệ là phải xóa. Project tránh cách tiếp cận đó bằng việc phân loại rõ trường nào bắt buộc, trường nào nullable và lỗi nào làm một bản ghi không thể sử dụng.

Các trường name, property_type_name và province_name được xem là bắt buộc vì chúng định danh và phân nhóm một tin đăng. area phải dương và hữu hạn vì đây là đại lượng vật lý phục vụ mô tả. published_at phải đọc được để hỗ trợ truy vấn thời gian. Trong khi đó, price có thể không được niêm yết mà tin đăng vẫn có giá trị địa lý; district_name và ward_name có thể trống do mức chi tiết nguồn khác nhau.

Quy tắc làm sạch tốt cần có tính quyết định: cùng đầu vào sẽ cho cùng đầu ra. Hàm dùng pandas để strip chuỗi, chuyển rỗng thành NA, ép kiểu số và thời gian, sau đó trả cả DataFrame và report đếm. Report giúp việc lọc không trở thành một thao tác “hộp đen”.

2.2 Xử lý theo luồng và tái lập dữ liệu

Dữ liệu lớn không nhất thiết phải được nạp hoàn toàn vào RAM để xử lý. Trong streaming, chương trình nhận dần bản ghi, gom theo batch, xử lý và ghi kết quả. Bộ nhớ chính được giới hạn bởi batch hiện tại và buffer shuffle, phù hợp khi nguồn có thể lớn hơn năng lực máy cục bộ. [2]

Tính tái lập của streaming cần đi kèm metadata. Seed 42, shuffle buffer 10.000, batch 2.000 và revision nguồn là những tham số có ảnh hưởng đến kết quả. Khi revision bị để null, builder vẫn chạy nhưng khả năng tái tạo đúng tập nguồn khi upstream thay đổi giảm. Vì vậy report phải lưu revision thay vì coi nó là chi tiết phụ.

Tệp nhỏ là prefix của tệp lớn khi sinh trong một lượt. Đây là thiết kế có ích cho benchmark vì việc tăng kích thước không thay đổi hoàn toàn mẫu dữ liệu ban đầu. Tuy nhiên, điều đó chỉ có ý nghĩa khi report cho biết các file thuộc cùng lượt completed.

2.3 Hệ quản trị cơ sở dữ liệu và tính nguyên tử

PostgreSQL là lớp lưu trữ và tối ưu hóa truy vấn của đồ án. Schema xác định kiểu dữ liệu, khóa chính và constraint; importer đưa dữ liệu vào theo transaction. Transaction bảo đảm tất cả thao tác quan trọng cùng thành công hoặc cùng thất bại. Nếu COPY lỗi ở giữa, các thay đổi chưa commit sẽ rollback thay vì để bảng mang một phần dữ liệu mới. [3], [4]

Khóa bảng SHARE ROW EXCLUSIVE giúp importer tránh cạnh tranh ghi có thể làm sai kiểm tra ban đầu. Tùy chọn replace đưa TRUNCATE vào cùng transaction. Đây là khác biệt quan trọng so với xóa bảng ở một lệnh riêng rồi import ở lệnh khác, bởi một lỗi giữa hai bước độc lập có thể để lại trạng thái không mong muốn.

COPY FROM STDIN phù hợp với CSV vì truyền dữ liệu theo block và không phải tạo hàng nghìn INSERT riêng lẻ. Mặc dù vậy, COPY chỉ là cơ chế nạp; tính đúng còn cần kiểm tra header, số trường, ID và NULL trước commit.

2.4 Chỉ mục B-tree và tối ưu truy vấn

B-tree là cấu trúc index cân bằng, đặc biệt hữu ích cho equality, range và ordering. Nó không phải một “nút tăng tốc” luôn có lợi. PostgreSQL sẽ ước lượng chi phí của index scan, bitmap scan hoặc sequential scan dựa trên thống kê, độ chọn lọc, số heap page cần đọc, cache và cấu hình planner.

Một index composite cần được thiết kế theo predicate và thứ tự. Với Q2, hai cột equality đứng đầu, tiếp theo là price range và id hỗ trợ thứ tự phụ. Với Q3, index published_at có thể sử dụng tốt hơn khi predicate là khoảng trên cột gốc thay vì cắt date từ timestamp. Các khả năng này là lý do để benchmark, không phải kết luận tốc độ đã được chứng minh. [3], [6]

Một index cũng có chi phí: dung lượng đĩa, thời gian tạo, chi phí cập nhật dữ liệu và đôi khi làm planner chọn plan không tối ưu trong một số phân bố. Vì thế benchmark cần ghi index_bytes và phải có query đối chứng tổng hợp toàn bảng.

2.5 Thiết kế thí nghiệm và chỉ số đánh giá

Bảng 2: Công nghệ và vai trò

Công nghệ	Vai trò trong project
Python	Điều phối build, cleaning, import và test
pandas	DataFrame, ép kiểu và thống kê
Hugging Face Datasets	Đọc nguồn ở chế độ streaming
PostgreSQL 18.3	Schema, COPY, index, EXPLAIN trong benchmark
psycopg 3	Kết nối và COPY giao dịch
pytest	Bảo vệ hồi quy offline

Đánh giá hiệu năng truy vấn phải tách warm-up khỏi lượt đo chính thức. Cache của database và hệ điều hành có thể làm lượt đầu chậm hơn các lượt sau. Hướng dẫn project yêu cầu ba warm-up, tiếp theo hai mươi lượt độc lập, dùng median Execution Time. Median ổn định hơn trung bình khi có ít giá trị bất thường lớn.

EXPLAIN ANALYZE thực thi query thật; BUFFERS cho biết shared hit/read/dirtied/written blocks; FORMAT JSON giúp giữ toàn bộ plan. Planning Time không được cộng vào Execution Time vì chúng là hai pha khác nhau. Khi diễn giải plan, cần xem cả node con vì bitmap hoặc parallel operation có thể không nằm ở node gốc.

Tính tương đương kết quả là điều kiện trước khi so sánh thời gian. Q3 dùng EXCEPT ALL theo hai chiều để kiểm tra query original và rewritten trả cùng tập hàng. Một query nhanh hơn nhưng trả khác kết quả không phải một tối ưu hợp lệ.

CHƯƠNG 3 KHẢO SÁT VÀ TIỀN XỬ LÝ DỮ LIỆU

Chương này sử dụng số liệu EDA của bộ 50.000 dòng chính thức, được khóa bằng SHA-256 `11bc4fe...da2f8b` trong manifest. Tập dữ liệu 50K trên workspace được xác thực mã băm SHA-256 trùng khớp hoàn toàn với manifest nghiệm thu. Các hình EDA được giữ để bảo toàn bố cục báo cáo; diễn giải số liệu trong phần văn bản được đối chiếu theo quality report chính thức.

Tập dữ liệu chuẩn hóa gồm chín cột. Đây là một schema hẹp hơn profile nguồn 5.000 dòng, vì nhiều trường bổ sung không được đưa vào pipeline benchmark. Sự giản lược có chủ đích giúp định nghĩa dữ liệu nạp vào PostgreSQL rõ ràng và tập trung vào các truy vấn được nghiệm thu.

3.1 Nguồn dữ liệu và lược đồ chuẩn hóa

Bảng 3: Lược đồ tập dữ liệu chuẩn hóa

Cột	Kiểu logic	Quy tắc chính
id	Số nguyên	Khóa liên tục sau lọc
name	Văn bản	Bắt buộc
property_type_name	Văn bản	Bắt buộc
province_name	Văn bản	Bắt buộc
district_name	Văn bản	Có thể NULL
ward_name	Văn bản	Có thể NULL
price	Số VND	Có thể NULL
area	Số thực	Dương, bắt buộc
published_at	Thời gian	Hợp lệ, bắt buộc

Nguồn khai báo trong builder là Phuscca/vietnam-real-estates, split train. CSV đầu ra dùng UTF-8 BOM; các giá trị NULL được ghi trường rỗng và timestamp theo định dạng đầy đủ đến microsecond. Các đặc điểm này được importer xử lý khi đọc lại và COPY.

ID không phải định danh nguyên gốc của nhà cung cấp. Với datasets lớn, ID được gán sau khi bản ghi vượt qua quy tắc làm sạch, liên tục từ 1 đến target. Điều này tạo một khóa kỹ thuật ổn định trong CSV nhưng không nên hiểu là định danh nghiệp vụ hay đại diện cho thứ tự thời gian của nguồn.

Lược đồ vừa đủ cho mục tiêu: có trường nhóm theo loại/tỉnh, trường địa chỉ tùy chọn, trường số cho giá/diện tích và trường thời gian. Mỗi cột được giải thích trong bảng để người đọc hiểu ràng buộc trước khi xem EDA.

3.1.1 Phạm vi dữ liệu được sử dụng trong báo cáo

Bộ dữ liệu tham chiếu của báo cáo là properties_50000.csv chính thức gồm 50.000 bản ghi, 9 thuộc tính và SHA-256 `11bc4fe...da2f8b`. Hai bộ 200K/500K được tạo trong cùng lượt build và là phần mở rộng theo tiền tố của 50K. Ba tập CSV đã được sinh tất định và lưu đầy đủ trong thư mục data/processed/ với mã băm SHA-256 trùng khớp hoàn toàn với manifest lưu trữ.

Nguồn ban đầu được project xác định là Phuscca/vietnam-real-estates, split train trên Hugging Face. Builder truy cập nguồn qua chế độ streaming rồi làm sạch theo batch. CSV cuối không phải bản sao thô của nguồn: nó chỉ giữ các thuộc tính phục vụ cho schema PostgreSQL và các truy vấn nghiệm thu. Do đó, các trường như description, street_name, project_name, floor_count hoặc bedroom_count có thể xuất hiện trong profile mẫu 5.000 dòng nhưng không thuộc hợp đồng dữ liệu 9 cột của pipeline lớn.

ID trong CSV là định danh kỹ thuật được gán sau khi một dòng vượt qua quy tắc làm sạch. Với build lớn, ID liên tục từ 1 đến target; nó không được dùng để suy luận thứ tự đăng tin hoặc ID nguyên gốc của nhà cung cấp. Sự phân biệt này quan trọng khi kiểm tra MIN(id), MAX(id) và COUNT(DISTINCT id) sau import: các kiểm tra đó chứng minh tính toàn vẹn của output pipeline, không chứng minh tính duy nhất ở hệ thống nguồn.

Bảng 4: Hợp đồng dữ liệu giữa CSV, pipeline và PostgreSQL

Thuộc tính	Ý nghĩa pipeline	Hệ quả khi nạp database
id	Gán sau lọc; liên tục 1..N	BIGINT PRIMARY KEY
name, type, province	Bắt buộc sau chuẩn hóa text	TEXT NOT NULL và không rỗng
district, ward	Có thể thiếu theo nguồn	TEXT nullable
price	Có thể thiếu hoặc không đọc được	NUMERIC nullable; chỉ giá dương giữ lại
area	Phải hữu hạn và lớn hơn 0	DOUBLE PRECISION NOT NULL, CHECK > 0
published_at	Phải đọc được ISO8601	TIMESTAMP NOT NULL

Từ điển dữ liệu này đồng thời là cơ sở giải thích các biểu đồ sau. Ví dụ, một tỷ lệ thiếu của price là đặc điểm được giữ lại có chủ đích; trong khi tỷ lệ thiếu của area sau xử lý bằng 0 vì area là điều kiện lọc bản ghi. Không thể diễn giải hai kết quả đó theo cùng một cách.

3.2 Kết quả build và làm sạch

Lượt build chính thức tạo đồng thời ba quy mô đã đọc 502.000 bản ghi từ stream shuffle; loại 203 dòng (202 thiếu trường bắt buộc, 1 diện tích không hợp lệ, 0 ngày lỗi) và xuất các target 50K, 200K, 500K. Với target 500K, 501.797 dòng hợp lệ đã được quan sát, trong đó 1.797 dòng hợp lệ còn lại của batch cuối không được ghi sau khi đạt target. Đây là hành vi dự kiến của xử lý theo batch, không phải dữ liệu bị mất không rõ nguyên nhân.

Module làm sạch đếm lý do lỗi độc lập. Vì vậy một dòng có thể tăng nhiều bộ đếm nguyên nhân nhưng chỉ tăng một ở removed_rows. Thiết kế đó làm báo cáo có thể giải thích: số điều kiện vi phạm không nhất thiết bằng số dòng bị loại. Trường price không hợp lệ được chuyển NULL thay vì làm mất dòng; trong lượt 50K, report ghi invalid_price_values bằng 0.

Kết quả cho thấy pipeline ưu tiên tính bảo toàn: chỉ những lỗi khiến bản ghi không thể dùng cho schema mới loại dòng. Các quyết định này phù hợp với mục tiêu truy vấn hơn là cố tạo phân phối "đẹp" cho mô hình dự báo.

3.2.1 Đọc đúng ý nghĩa các bộ đếm trong build report

Build report chính thức ghi source_rows_read = 502.000, invalid_rows = 203, valid_rows_seen = 501.797 và exported_rows = 500.000 đối với target lớn nhất. Ba target 50K/200K/500K là các prefix cùng lượt build; chỉ số NULL riêng của 50K được lấy từ output chính thức, không từ lượt build local 52.000 dòng.

valid_rows_seen lớn hơn exported_rows 1.797 dòng ở target 500K vì builder xử lý theo batch. Khi target được chạm trong batch cuối, các dòng hợp lệ còn lại của batch đó không được ghi ra file. Đây là hành vi đúng theo mục tiêu tạo đúng kích thước output và được report rõ qua valid_rows_not_exported.

Trong lượt build chính thức, invalid_required = 202, invalid_area = 1 và invalid_date = 0. invalid_price_values = 0 không có nghĩa toàn bộ dòng có price; nó chỉ nói không có giá đã đọc thành số nhưng không hữu hạn hoặc không dương. Giá bị thiếu hoặc không đọc được trở thành NULL và vẫn thuộc output nếu các trường bắt buộc khác hợp lệ.

Bảng 5: Diễn giải các chỉ số của lượt build chung ba quy mô 50K, 200K và 500K

Chỉ số	Giá trị	Diễn giải đúng
source_rows_read	502.000	Record đã được đưa vào build chính thức 3 target
invalid_rows	203	202 thiếu trường bắt buộc, 1 diện tích không hợp lệ
valid_rows_seen	501.797	Dòng hợp lệ đã gặp trước giới hạn target 500K
valid_rows_not_exported	1.797	Dòng hợp lệ dư trong batch cuối của target 500K
exported_rows	50K / 200K / 500K	Ba output chính thức, lồng nhau theo tiền tố
invalid_price_values	0	Không có giá số <= 0 hoặc không hữu hạn

Việc báo cáo riêng các bộ đếm là một lợi thế kiểm toán. Nếu chỉ lưu số dòng cuối cùng, người đọc không biết dữ liệu giảm vì thiếu text, area sai hay date sai. Trong pipeline này, source code tính từng nguyên nhân trước khi filter nên những nguyên nhân có thể chồng lấp; ngược lại invalid_rows chỉ tính mỗi dòng một lần.

3.3 Dữ liệu thiếu sau chuẩn hóa



Hình 1: Mức độ thiếu của các trường quan trọng trong CSV 50.000 dòng

Trên 50.000 dòng chính thức, price thiếu 3.993 lần (7,99%), district_name thiếu 1.507 lần (3,01%) và ward_name thiếu 7.277 lần (14,55%). Các cột bắt buộc sau làm sạch

đều đầy đủ. Mức độ thiếu khác nhau theo trường cho thấy không nên áp dụng một quy tắc thay thế chung cho mọi cột.

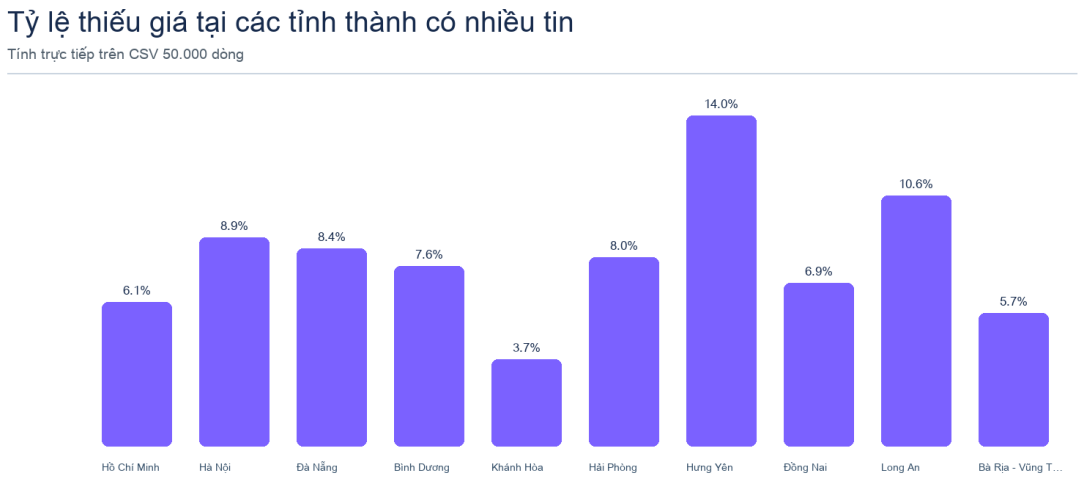
Giá thiếu được giữ NULL để tránh biến một tin không công khai giá thành một giá trị nhân tạo. Trong SQL, WHERE price BETWEEN ... sẽ bỏ những dòng này vì so sánh với NULL cho kết quả unknown; AVG(price) cũng bỏ NULL; nhưng COUNT(*) vẫn đếm chúng. Người đọc cần giữ sự khác biệt đó khi xem Q2 và Q4.

Địa chỉ thiếu không bị suy đoán bằng giá trị của tỉnh. Bổ sung quận/huyện hoặc phường/xã chỉ hợp lý nếu có quy tắc nguồn đáng tin cậy; project không có bảng ánh xạ địa lý nên không thực hiện bước đó.

3.3.1 Mẫu thiếu theo địa lý và ý nghĩa phân tích

Trong CSV 50K chính thức, price thiếu 3.993 dòng (7,99%), district_name thiếu 1.507 dòng (3,01%) và ward_name thiếu 7.277 dòng (14,55%). Các tỷ lệ này không được xem là lỗi còn sót lại của cleaning; chúng là thông tin về mức độ hoàn chỉnh khác nhau của tin đăng sau khi pipeline đã bảo đảm các thuộc tính bắt buộc.

Tỷ lệ thiếu price không giống nhau hoàn toàn giữa các tỉnh/thành có nhiều tin. Sự khác nhau này chỉ là mô tả của tập CSV, không phải bằng chứng rằng một địa phương "kém minh bạch" hơn địa phương khác: nó có thể chịu ảnh hưởng của loại bất động sản, chính sách đăng tin, thời điểm thu thập hoặc cách ghi của nguồn. Bởi vậy, biểu đồ dưới đây được dùng để kiểm tra mức độ thiếu có cấu trúc, không để đưa ra kết luận nhân quả.

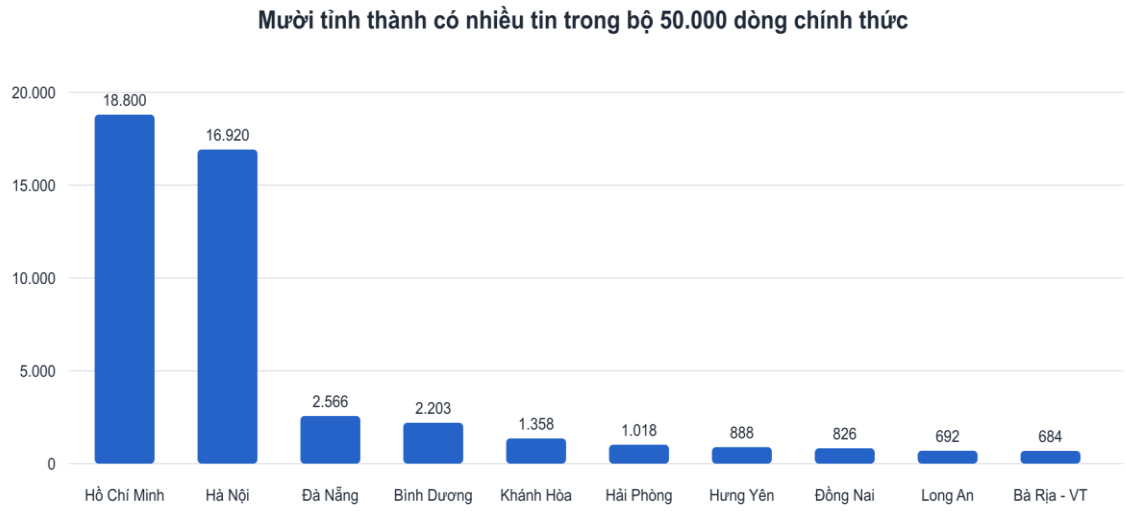


Hình 2: Tỷ lệ thiếu giá tại mười tỉnh thành có nhiều tin nhất

Trong các phân tích tiếp theo, cần nêu rõ tập con được sử dụng. Thống kê giá, median giá và biểu đồ phân phối price chỉ áp dụng cho các dòng có price hợp lệ; thống kê số tin, tỉnh, loại và diện tích áp dụng cho toàn bộ 50.000 dòng. Nêu rõ mẫu số là cách tránh diễn giải nhầm một average trên tập con thành đặc trưng của toàn bộ tập dữ liệu.

Project không thực hiện imputation. Thay giá thiếu bằng trung bình, median hoặc giá theo tỉnh sẽ tạo ra những giá trị chưa từng tồn tại trong nguồn và có thể làm sai Q2 hoặc AVG(price). Với phạm vi hiện tại, bảo toàn NULL là lựa chọn có bằng chứng tốt hơn; imputation chỉ nên được thêm khi có câu hỏi phân tích, quy tắc và bước đánh giá riêng.

3.4 Phân bố theo tỉnh thành



Hình 3: Mười tỉnh thành có nhiều tin đăng nhất

Sự phân bố tin đăng theo tỉnh thành không đồng đều. Điều này có ảnh hưởng trực tiếp đến việc đánh giá Q1: nếu điều kiện tỉnh trả về một phần lớn bảng, index một cột có thể không thắng quét tuần tự; nếu điều kiện chọn lọc hơn, index có cơ hội hữu ích hơn. Bởi vậy, không thể chỉ nhìn định nghĩa index để kết luận hiệu năng.

CSV có 62 giá trị province_name khác nhau. Biểu đồ hiển thị mười địa phương có nhiều tin nhất nhằm mô tả cấu trúc dữ liệu, không nhằm xếp hạng thị trường hay suy ra nhu cầu bất động sản. Dataset là tập tin đăng, không phải điều tra đại diện cho dân số hoặc giao dịch thực.

Việc chuẩn hóa hiện chỉ strip khoảng trắng; nó không hợp nhất các cách viết địa danh. Đây là hạn chế cần ghi nhận khi dùng equality predicate. Một giai đoạn chuẩn hóa địa lý sâu hơn có thể được đề xuất, nhưng không được mô tả như tính năng đã hoàn thành.

3.4.1 Mức độ tập trung không gian của tập tin đăng

Phân bố theo province_name tập trung mạnh ở hai đô thị lớn. Hồ Chí Minh có 18.800 tin, Hà Nội có 16.920 tin; hai nhãn này chiếm phần lớn CSV chính thức. Các tỉnh tiếp theo như Đà Nẵng, Bình Dương, Khánh Hòa và Hải Phòng có quy mô nhỏ hơn rõ rệt. Đây là đặc trưng của tập tin đăng được thu thập, không phải kết quả thống kê về tổng cung bất động sản của toàn quốc.

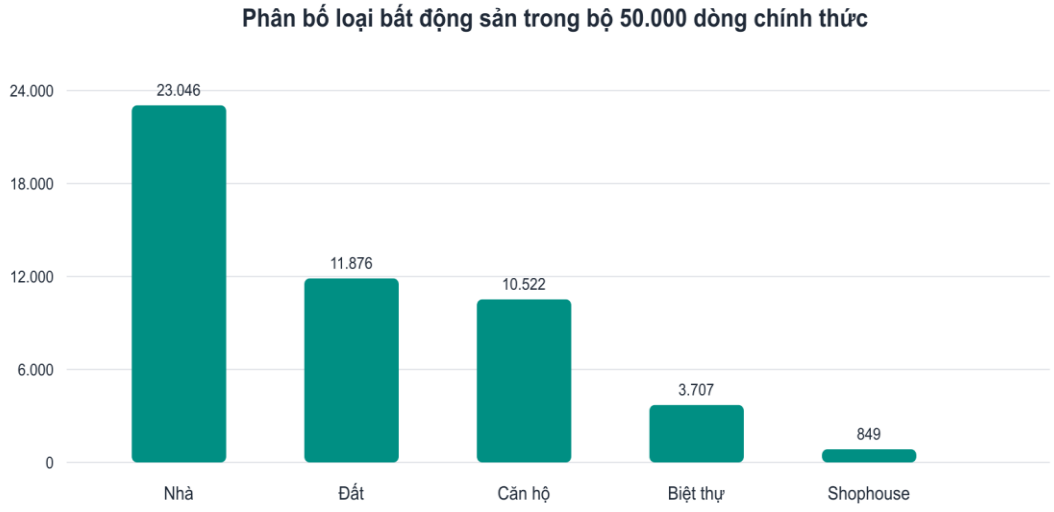
Sự tập trung này tạo hệ quả trực tiếp cho benchmark. Điều kiện province_name = Hải Phòng trong bộ 50K chính thức trả 1.018 dòng theo execution plan đã lưu, trong khi một điều kiện Hà Nội sẽ có mức chọn lọc khác. Planner PostgreSQL không chỉ nhìn tên cột có index mà còn ước lượng số dòng và số page phải đọc. Vì vậy, kết quả Q1 không được khái quát cho mọi địa phương dù cùng dùng một index đơn cột.

Bảng 6: Mười tỉnh thành có nhiều tin nhất trong CSV 50K

Tỉnh/thành	Số tin	Tỷ trọng trong CSV
Hồ Chí Minh	18.800	37,60%
Hà Nội	16.920	33,84%
Đà Nẵng	2.566	5,13%
Bình Dương	2.203	4,41%
Khánh Hòa	1.358	2,72%
Hải Phòng	1.018	2,04%
Hưng Yên	888	1,78%
Đồng Nai	826	1,65%
Long An	692	1,38%
Bà Rịa - Vũng Tàu	684	1,37%

Địa chỉ hiện được chuẩn hóa ở mức strip khoảng trắng và chuyển chuỗi rỗng thành NA. Pipeline không ánh xạ “TP HCM”, “Hồ Chí Minh” hoặc các biến thể có dấu/không dấu về cùng một giá trị. Vì vậy, nếu dữ liệu nguồn tương lai có biến thể địa danh, một lớp chuẩn hóa danh mục phải được thiết kế, kiểm thử và báo cáo riêng thay vì âm thầm áp dụng trong truy vấn.

3.5 Loại bất động sản và truy vấn tổng hợp



Hình 4: Phân bố loại bất động sản theo nhãn nguồn

property_type_name có 5 nhãn khác nhau trong CSV. Các nhãn được giữ theo nguồn, không gộp bằng quy tắc thủ công. Cách bảo toàn này hữu ích khi Q4 GROUP BY, vì kết quả phản ánh đúng taxonomy hiện có; đổi lại có thể tồn tại biến thể nhãn mà pipeline không đồng nhất.

Q4 là tình huống tổng hợp toàn bảng, tính COUNT(*) và AVG(price) theo loại. Nó được dùng như đối chứng trong benchmark: index trên property_type_name có thể không đem lại giảm thời gian vì executor vẫn cần đọc giá để tính trung bình. Tính đối chứng giúp báo cáo không rơi vào thiên kiến chỉ quan sát kịch bản thành công.

Các nhóm có cùng total có thể có thứ tự khác giữa lần chạy vì query chỉ ORDER BY total DESC. Khi kiểm tra tương đương, phải map theo property_type_name thay vì coi khác biệt thứ tự hiển thị là khác biệt dữ liệu.

3.5.1 Quy mô từng nhóm và khả năng diễn giải Q4

property_type_name có năm nhãn trong file 50K: Nhà, Đất, Căn hộ chung cư, Biệt thự/Nhà liền kề và Shophouse. Đây là số lượng nhãn thực tế của CSV, không phải taxonomy đầy đủ của thị trường. Các nhãn được giữ nguyên theo nguồn để Q4 có thể phản ánh đúng giá trị đã nhập thay vì một phép gộp không có quy tắc chứng minh.

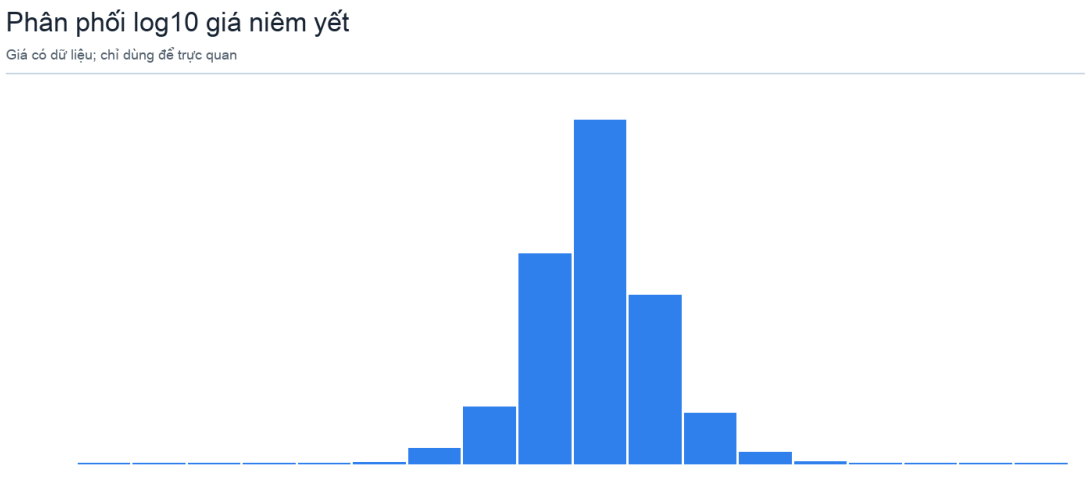
Bảng 7: Quy mô nhóm property_type_name

Loại bất động sản	Số tin	Tỷ trọng
Nhà	23.046	46,09%
Đất	11.876	23,75%
Căn hộ chung cư	10.522	21,04%
Biệt thự/Nhà liền kề	3.707	7,41%
Shophouse	849	1,70%

Nhóm Nhà có 23.046 tin và là nhóm lớn nhất; Đất, Căn hộ chung cư, Biệt thự/Nhà liền kề và Shophouse có quy mô giảm dần. Khi Q4 trả COUNT(*) theo nhóm, giá trị total chịu ảnh hưởng trực tiếp bởi phân bố này. Khi Q4 trả AVG(price), số quan sát hợp lệ cho giá lại có thể khác total do price NULL; do đó người đọc không nên so sánh average mà quên bối cảnh quy mô nhóm.

Query Q4 được thiết kế như một đối chứng hiệu năng. Bởi query phải tổng hợp trên toàn bảng, index property_type_name có thể không làm giảm thời gian hoặc không được planner chọn. Lợi ích của kịch bản này là buộc quá trình benchmark ghi nhận kết quả không thuận lợi cho index với cùng mức nghiêm túc như các kịch bản có thể cải thiện.

3.6 Phân phối giá niêm yết



Hình 5: Phân phối log10 của giá niêm yết

Giá trung vị của các dòng có price là 7.400.000.000 VND, trong khi giá trung bình là 15.801.406.058 VND (khoảng 15,80 tỷ VND). Sự chênh giữa hai thống kê là lý do cần

báo cáo cả hai: trung bình nhạy với các giá trị lớn, còn trung vị mô tả điểm giữa của dữ liệu có giá. Không thống kê nào trong hai số này đại diện cho các dòng price NULL.

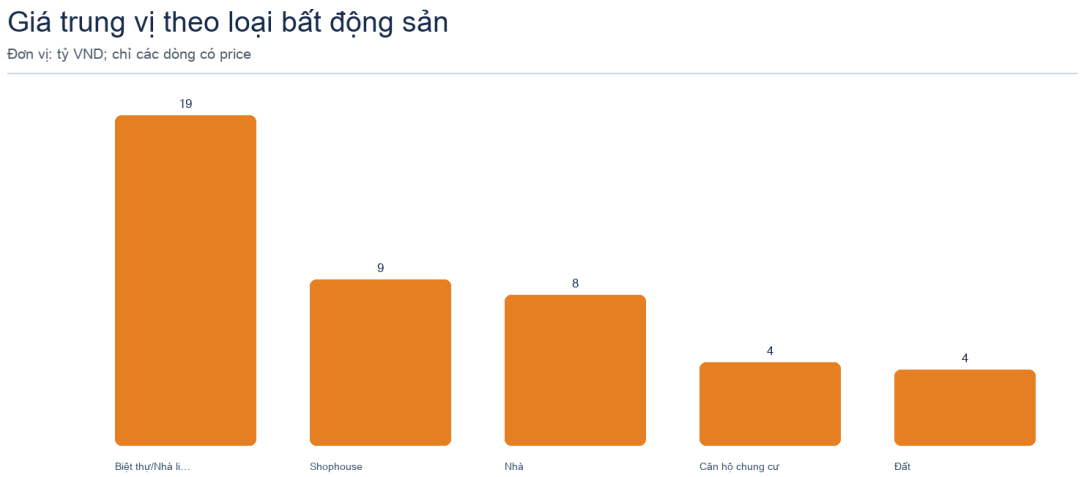
Hình dùng log10(VND) để tạo thang nhìn được khi giá trải trên miền rộng. Phép log chỉ phục vụ trực quan hóa; cột price trong CSV, schema và Q2 vẫn dùng đơn vị VND gốc. Bảo toàn đơn vị ở tầng query là cần thiết để điều kiện BETWEEN có ý nghĩa rõ ràng.

Dữ liệu giá không được dùng để huấn luyện mô hình trong project. Vì vậy, các biểu đồ phân phối được diễn giải như EDA, không như kiểm tra residual hoặc feature distribution của một pipeline ML.

3.6.1 Phân vị, đuôi phân phối và so sánh theo loại

Giá hợp lệ có median 7.400.000.000 VND, Q1 4.150.000.000 VND và Q3 14.300.000.000 VND. Phân vị 95% là 48.000.000.000 VND (48 tỷ VND), trong khi phân vị 99% là 128.000.000.000 VND (128 tỷ VND). Miền giá rất rộng: min là 1.100.000 VND và max là 23.350.000.000.000 VND (23,35 nghìn tỷ VND). Điều này giải thích tại sao biểu đồ giá dùng thang log10 để quan sát phần đông dữ liệu mà không làm mất khả năng nhìn thấy các nhóm giá thấp/trung bình.

Khoảng cách giữa trung bình và trung vị là tín hiệu dữ liệu lệch phải, nhưng không tự chứng minh các giá trị cao là lỗi. Giá niêm yết bất động sản chịu ảnh hưởng lớn của loại, vị trí, diện tích và cách người đăng định giá. Pipeline không có quy tắc cắt ngoại lệ giá; mọi nhận định về “bất thường” ở báo cáo chỉ là điểm cần kiểm tra nghiệp vụ, không phải tiêu chí xóa đã được implement.

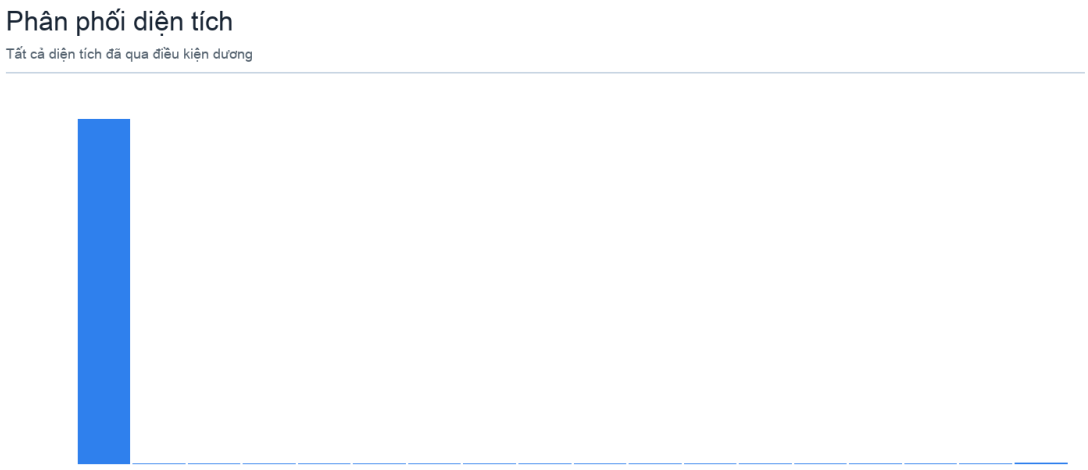


Hình 6: Giá trung vị theo loại bất động sản; đơn vị tỷ VND

Biểu đồ theo nhóm sử dụng median thay vì mean để giảm chi phối của đuôi phải khi mô tả giá điển hình của các dòng có price. Kết quả là EDA bổ sung được tính trực tiếp từ CSV; nó không phải feature engineering có mặt trong source code và không được ghi trở lại CSV. Việc tách hai mục đích này giúp báo cáo không lẫn lộn phân tích với thay đổi dữ liệu sản xuất.

Q2 sử dụng khoảng giá 4,1 đến 14,3 tỷ VND, bao gồm hai biên. Các phân vị cho thấy khoảng này nằm trong phần trung tâm của price có dữ liệu, nhưng số bản ghi thỏa điều kiện còn phụ thuộc đồng thời vào province_name và district_name. Do đó không thể suy từ histogram giá sang số hàng Q2 mà không tính đủ các predicate.

3.7 Phân phối diện tích và giá trị ngoại biên



Hình 7: Phân phối diện tích trong tập dữ liệu đã làm sạch

Diện tích trung vị là 80,00 m² và trung bình là 258,81 m². Pipeline chỉ yêu cầu area là số hữu hạn lớn hơn 0. Không có quy tắc loại outlier dựa trên percentile, IQR hay ngưỡng cứng. Việc không cắt dữ liệu thể hiện nguyên tắc rằng một diện tích lớn chưa chắc là lỗi.

Biểu đồ diện tích cho phép phát hiện sự lệch hoặc đuôi dài, nhưng không tự gán nguyên nhân. Một điểm ở đuôi có thể là bất động sản thật, đơn vị nhập khác, hoặc lỗi nguồn. Muốn sửa nó phải có tài liệu domain hoặc kiểm chứng nguồn, không chỉ dựa vào trực quan hóa.

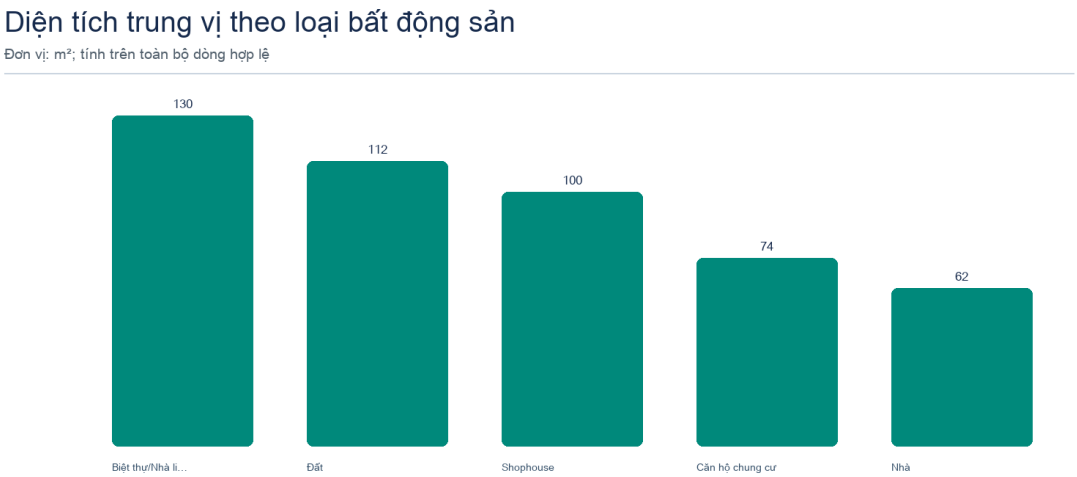
Ràng buộc CHECK area > 0 ở PostgreSQL lặp lại điều kiện của cleaning. Phòng vệ ở hai tầng này giúp bảo vệ database ngay cả khi CSV được tạo bởi một tiến trình khác trong tương lai.

3.7.1 Đặc trưng diện tích theo mức phân vị và loại

Diện tích có Q1 56,00 m², median 80,00 m² và Q3 120,00 m². Đuôi trên rộng: phân vị 95% là 383,00 m² và phân vị 99% là 2.167,30 m². Giá trị lớn nhất 440.000,00 m² làm trung bình bị kéo lên đáng kể so với median, đặc biệt ở những nhóm có một số quan sát diện tích rất lớn.

Quy tắc implemented chỉ khẳng định area phải hữu hạn và dương. Nó không nói rằng diện tích cần nằm trong một khoảng "hợp lý" cụ thể. Đây là lựa chọn thận trọng vì ngưỡng hợp lý phụ thuộc loại bất động sản và đơn vị nguồn. Tự đặt ngưỡng 500 m²

hoặc 1.000 m² trong báo cáo sẽ là thay đổi chính sách, không phải mô tả đúng source code.



Hình 8: Diện tích trung vị theo loại bất động sản

Khi phân tích theo loại, đất và biệt thự/nhà liền kề có median diện tích lớn hơn căn hộ và nhà trong CSV này. Kết quả giúp diễn giải vì sao quan hệ price-area không thể giản lược thành một đường tuyến tính chung cho toàn bộ tập. Các nhóm khác nhau có cấu trúc giá và diện tích khác nhau; project hiện chỉ mô tả chúng, không fit mô hình phân tầng.

Ở tầng database, constraint CHECK area > 0 bảo vệ cùng invariant với cleaning. Nếu một CSV không qua builder vẫn được đưa vào importer, database có thêm tuyến phòng thủ. Tuy nhiên, constraint không phát hiện các giá trị dương nhưng không hợp lý về nghiệp vụ; kiểm soát đó cần một yêu cầu domain rõ ràng.

3.8 Quan hệ mô tả giữa diện tích và giá



Hình 9: Quan hệ mô tả giữa diện tích và giá niêm yết

Hệ số Spearman trên các cặp price-area hợp lệ là 0,276. Hệ số này mô tả mức độ cùng biến thiên theo thứ hạng; nó không chứng minh diện tích là nguyên nhân làm giá thay đổi. Giá còn bị chi phối bởi địa điểm, loại bất động sản, thời gian và các biến không có trong CSV chuẩn.

Hình scatter hiển thị mẫu 3.500 điểm và cắt 1% đuôi trên của hai biến chỉ để tránh hiện tượng vài điểm che khuất toàn bộ đồ thị. Việc cắt này không được áp dụng cho dữ liệu dùng thống kê hoặc query. Chú thích như vậy cần thiết để người đọc không hiểu nhầm sự thay đổi cách hiển thị là bước xử lý dữ liệu.

Nếu phát triển thành bài toán dự báo, cần định nghĩa nhãn, xử lý price NULL, chọn feature và chia tập theo thời gian. Không có những thành phần đó trong source code, nên báo cáo dừng ở phân tích mô tả.

3.8.1 Cách đọc tương quan mà không suy diễn nhân quả

Hệ số tương quan Spearman giữa price và area trên các cặp có cả hai giá trị là 0,276. Spearman được dùng trong EDA vì nó mô tả xu hướng thứ hạng đơn điệu mà không yêu cầu mối quan hệ tuyến tính hoặc phân phối chuẩn. Dù vậy, đây vẫn chỉ là tóm tắt hai biến, không thay thế phân tích đa biến.

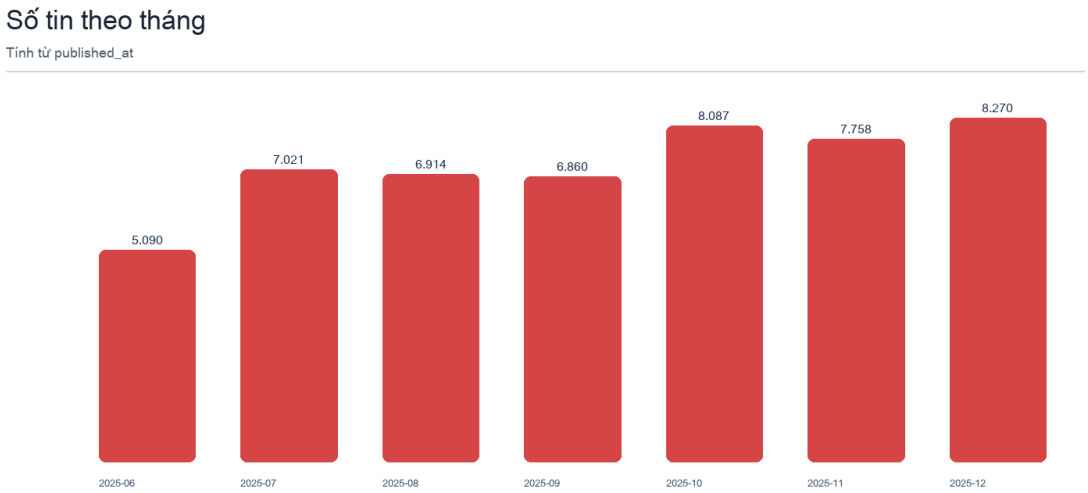
Quan sát một xu hướng cùng chiều không cho phép khẳng định diện tích gây ra giá. Province, district, loại bất động sản, thời gian đăng và các thông tin không có trong CSV có thể ảnh hưởng đồng thời đến hai đại lượng. Hơn nữa, price bị thiếu ở một

phần dữ liệu, nên phép tính tương quan chỉ dùng tập con có giá; nếu cơ chế thiếu có cấu trúc, tập con này không nhất thiết đại diện hoàn toàn cho mọi tin đăng.

Đồ thị scatter trong báo cáo cắt 1% đuôi trên và lấy mẫu để tránh chồng điểm. Cắt/gom chỉ là thao tác trực quan hóa, không thay đổi file và không được dùng trong Q1-Q4. Chú thích ranh giới này giúp người đọc phân biệt giữa dữ liệu được lưu, dữ liệu được query và dữ liệu được hiển thị.

Nếu mở rộng sang dự báo giá, cần xác định mục tiêu, chính sách price NULL, cách mã hóa category, cách chia train/validation/test theo thời gian và metric ngoài mẫu. Vì project không có các artefact đó, báo cáo không gán ý nghĩa predictive cho EDA hiện tại.

3.9 Dấu thời gian và tính sẵn sàng cho Q3



Hình 10: Số tin đăng theo tháng

Thời gian đăng trong CSV chạy từ 2025-06-01 00:36:24.154000 đến 2025-12-30 23:09:00.381000. Biểu đồ số tin theo tháng xác nhận published_at được đọc như biến thời gian và cho thấy dữ liệu không phân bố đều theo tháng. Mức độ tập trung theo ngày là một yếu tố quyết định độ chọn lọc của Q3.

Trên tập 50K, điều kiện ngày 15/10/2025 phù hợp 287 dòng theo phép đếm pandas. Số này dùng để kiểm tra logic dữ liệu trước import, không phải thời gian chạy PostgreSQL. Sau khi nạp, SQL original và rewritten phải kiểm chứng cùng tập hàng trước khi benchmark.

Q3 dùng khoảng nửa mở từ đầu ngày đến đầu ngày tiếp theo. Cách viết tránh bao gồm nhầm nửa đêm của ngày sau, đồng thời không bọc cột published_at trong phép biến đổi date. Đây là một thay đổi biểu thức cần đo, không phải một kết luận được xem là đúng trong mọi index.

3.9.1 Mật độ dữ liệu thời gian và kiểm tra ngữ nghĩa query

published_at trải từ 2025-06-01 00:36:24.154000 đến 2025-12-30 23:09:00.381000 và có 213 ngày lịch khác nhau. Số tin theo tháng thay đổi từ 5.090 đến 8.270 tin. Sự thay đổi này là lý do không thể giả định điều kiện một ngày bất kỳ luôn có cùng độ chọn lọc trong Q3.

Bảng 8: Số tin theo tháng đăng

Tháng	Số tin
2025-06	5.090
2025-07	7.021
2025-08	6.914
2025-09	6.860
2025-10	8.087
2025-11	7.758
2025-12	8.270

Q3 chọn ngày 15/10/2025. Query original dùng published_at::date = DATE; query rewritten dùng khoảng [2025-10-15 00:00:00, 2025-10-16 00:00:00). Hai cách viết chỉ có thể được so sánh sau khi kiểm tra kết quả hai chiều bằng EXCEPT ALL. So count bằng nhau là chưa đủ vì hai tập hàng khác nhau vẫn có thể có cùng kích thước.

Schema dùng TIMESTAMP WITHOUT TIME ZONE. Vì vậy, report chỉ diễn giải ngày theo giá trị đã lưu, không thêm giả định về chuyển đổi múi giờ. Nếu nguồn có dữ liệu từ nhiều múi giờ, cách lưu và cách query thời gian phải được thiết kế lại ở cấp schema; đây không phải vấn đề mà Q3 hiện có đã giải quyết.

CHƯƠNG 4 THIẾT KẾ PIPELINE VÀ CƠ SỞ DỮ LIỆU



Hình 11: Pipeline dữ liệu và đánh giá hiệu năng của project

Chương 4 chuyển từ quan sát dữ liệu sang mô tả hiện thực. Pipeline được xem như chuỗi trách nhiệm có điểm kiểm soát: nguồn, làm sạch, xuất CSV, import, index và đo. Mỗi bước đều có artifact hoặc code có thể kiểm tra; không có bước nào yêu cầu người đọc tin vào một kết quả chưa được lưu.

Sơ đồ tổng quát giúp liên hệ những module riêng lẻ. Phần nguồn và cleaning tạo ra dữ liệu có cấu trúc; phần COPY đưa cấu trúc đó vào database; phần benchmark dùng database để đo query. Mọi thay đổi ở một lớp cần được phản ánh trong kiểm chứng của lớp sau.

4.1 Hiện thực làm sạch dùng chung

clean_dataframe() nhận DataFrame nguồn và kiểm tra có đủ tám cột kỳ vọng hay không. Hàm sau đó tạo copy, do đó không sửa trực tiếp dữ liệu đầu vào của caller. Các trường text được ép kiểu string, loại khoảng trắng đầu/cuối và chuyển chuỗi rỗng thành NA. Việc thống nhất logic trong một hàm tránh tình trạng script mẫu và builder lớn áp dụng hai chính sách khác nhau.

Đối với số, pandas.to_numeric(errors="coerce") tạo NaN khi không thể chuyển. Giá không hữu hạn hoặc không dương bị chuyển NULL. Diện tích NaN, không hữu hạn hoặc không dương tạo invalid_area. published_at được đọc ISO8601 với errors="coerce". Các điều kiện loại được OR lại, sau đó report lưu original_rows, removed_rows, cleaned_rows và các counter liên quan.

clean_data.py dùng cùng hàm cho mẫu 5.000 dòng và giữ source index làm ID. Builder lớn dùng hàm trong batch và gán ID mới sau lọc. Hai cách gán ID khác nhau phản ánh mục tiêu của hai workflow, nhưng quy tắc data quality vẫn giống nhau.

4.1.1 Trình tự chuyển đổi dữ liệu trong clean_dataframe

Hàm clean_dataframe trước hết xác nhận input có đủ tám cột nguồn mong đợi. Sau đó hàm lấy một bản sao chỉ gồm các cột trong hợp đồng. Bước copy có ý nghĩa thực tế: caller có thể tiếp tục dùng DataFrame gốc để kiểm tra hay gắn ID mà không bị thay đổi ngầm bởi cleaning. Đây là một đặc tính thuận lợi khi unit test và khi tái sử dụng module trong nhiều script.

Năm cột text gồm name, property_type_name, province_name, district_name và ward_name được ép sang pandas StringDtype, strip khoảng trắng rồi đổi chuỗi rỗng sang NA. Làm sạch whitespace trước khi kiểm tra trường bắt buộc tránh trường hợp một ô chứa nhiều dấu cách bị coi là tên hoặc tỉnh hợp lệ. Đối với địa chỉ tùy chọn, cùng thao tác này bảo đảm chuỗi trống được lưu nhất quán như NULL thay vì nhiều biểu diễn khác nhau.

Đối với price và area, to_numeric(errors="coerce") đưa những biểu diễn không đọc được về giá trị thiếu. Price không hữu hạn hoặc <= 0 bị đổi NA; area NA, không hữu hạn hoặc <= 0 tạo invalid_area. published_at được đọc với format ISO8601 và errors="coerce". Sau cùng, invalid_required, invalid_area và invalid_date được hợp lại thành điều kiện loại dòng.

Điểm đáng chú ý là report được tính trước khi lọc. Nhờ đó, một dòng vừa thiếu province vừa sai area có thể xuất hiện ở hai counter lý do nhưng chỉ làm removed_rows tăng một. Cách làm này vừa bảo tồn thông tin chẩn đoán vừa duy trì định nghĩa rõ ràng cho tổng số dòng loại.

4.2 Xây dựng dataset lớn và công bố an toàn

Builder đọc stream một lần, tạo các file tạm cạnh đường dẫn đích và ghi từng batch. Khi một batch có các dòng hợp lệ, code chỉ ghi số dòng cần thiết để lấp target. Sau khi đạt target lớn nhất, các tệp tạm được đóng và đổi tên; report JSON được publish cuối cùng. Điều này giúp người dùng không nhầm file đang xây với output đã hoàn tất.

Nếu thiếu bản ghi, network lỗi, ghi lỗi hoặc Ctrl+C, finally cố xóa .part và các file đã public trong lượt đó. Một lỗi cleanup được báo riêng nhưng không che lỗi gốc. Thiết

kế này không thể loại trừ mọi rủi ro mất điện cưỡng bức, nhưng làm trạng thái bình thường có tính nhất quán tốt hơn việc ghi trực tiếp file đích.

Các counter valid_rows_not_exported và total_csv_rows_written có nghĩa khác nhau. Counter đầu chỉ các dòng hợp lệ dư trong batch cuối; counter sau là tổng số dòng ghi trên tất cả target, nên có thể lớn hơn source rows do một dòng được ghi vào nhiều file prefix.

4.2.1 Giới hạn bộ nhớ, file tạm và tính toàn vẹn publish

build_large_dataset.py không đọc toàn bộ source vào bộ nhớ. Generator iter_batches cắt stream thành list tối đa batch_size record; batch được biến thành DataFrame, làm sạch rồi ghi dần. Bộ nhớ có thể chịu ảnh hưởng bởi shuffle buffer của thư viện và batch hiện tại, nhưng không phụ thuộc trực tiếp vào toàn bộ số record trong nguồn như cách đọc DataFrame hoàn chỉnh.

Mỗi target được ghi vào NamedTemporaryFile với hậu tố .csv.part ngay trong thư mục output. Các handle được quản lý bằng ExitStack để đóng đầy đủ trước khi đổi tên. Sau khi đủ mọi target, code kiểm tra lại đường dẫn đích chưa tồn tại, rồi rename staged CSV và cuối cùng rename report JSON. Thứ tự này khiến report completed chỉ xuất hiện khi các output đã được công bố.

Nếu xảy ra lỗi, cleanup cố xóa các file tạm và những file đã public trong chính lượt đó theo thứ tự ngược. Code chủ động không ghi đè output cũ; một lần chạy lại cùng target yêu cầu người vận hành di chuyển kết quả trước. Chính sách từ chối ghi đè bảo vệ against việc vô tình thay thế dữ liệu đã được benchmark bằng một build khác có seed/revision khác.



Hình 12: Chuỗi kiểm soát khi nạp CSV vào PostgreSQL

Số dòng được xuất vào các target nhiều kích thước không được diễn giải như số record nguồn duy nhất. Với ba target 50K, 200K, 500K, total_csv_rows_written là tổng 750K do prefix được ghi vào nhiều file. exported_rows là số dòng duy nhất được xuất tại target lớn nhất. Báo cáo cần giữ hai chỉ số riêng để không tạo mâu thuẫn giả tạo.

4.3 Schema PostgreSQL và ràng buộc

Bảng 9: Ràng buộc schema PostgreSQL

Thành phần	Ràng buộc	Vai trò
id	PRIMARY KEY	Duy nhất và không NULL; importer/validator kiểm tra tính liên tục
name/type/province	NOT NULL, BTRIM <> rỗng	Bảo đảm phân loại được
price	NUMERIC, CHECK > 0	Giữ NULL hoặc giá dương
area	DOUBLE PRECISION, CHECK > 0	Loại diện tích vô nghĩa
published_at	TIMESTAMP NOT NULL	Hỗ trợ lọc theo ngày

Bảng public.properties có id BIGINT PRIMARY KEY, ba text field không rỗng, hai trường địa chỉ nullable, price NUMERIC, area DOUBLE PRECISION và published_at TIMESTAMP WITHOUT TIME ZONE. Các constraint CHECK bảo vệ price và area dương, đồng thời BTRIM của ba text bắt buộc không rỗng. Như vậy database là lớp kiểm tra thứ hai sau Python.

Chọn NUMERIC cho price giúp tránh một số vấn đề biểu diễn số thực khi lọc các ngưỡng VND. Chọn DOUBLE PRECISION cho area phù hợp với schema hiện hữu và điều kiện dương. Sự lựa chọn schema là một hợp đồng: importer kiểm tra cột database đúng thứ tự mong đợi trước khi COPY.

Timestamp không gắn timezone trong schema, do đó query Q3 so sánh theo lịch của giá trị đã lưu. Nếu hệ thống tương lai cần dữ liệu từ nhiều múi giờ, đây là một điểm phải thiết kế lại thay vì tự suy diễn trong query.

4.3.1 Ánh xạ từ schema CSV sang schema quan hệ

Schema có chín cột theo đúng thứ tự importer kiểm tra. id là khóa chính; name, property_type_name và province_name vừa NOT NULL vừa có CHECK BTRIM <> rỗng. Việc có cả hai ràng buộc là hợp lý vì NOT NULL không ngăn chuỗi chỉ có whitespace, còn BTRIM kiểm tra trường hợp đó ở tầng database.

price là NUMERIC và nullable. CHECK (price > 0) vẫn cho phép NULL vì điều kiện CHECK chỉ từ chối khi biểu thức là false; giá trị unknown từ NULL không vi phạm. Đây là hành vi phù hợp với ý định từ cleaning: giá không có thông tin được giữ là thiếu, còn giá dương được chấp nhận. area thì NOT NULL kết hợp CHECK > 0 vì bản ghi thiếu diện tích đã bị loại trước đó.

published_at dùng TIMESTAMP WITHOUT TIME ZONE. Kiểu này phù hợp với output textual đang có, nhưng không tự ghi thông tin timezone. Điều này lý giải tại sao benchmark Q3 chỉ xác định ngày bằng timestamp literal; report không được suy diễn rằng query đã xử lý chuyển đổi timezone hoặc daylight-saving.

Bảng 10: Kiểm tra song song giữa pipeline và database

Quy tắc	Trong cleaning	Trong PostgreSQL
Text bắt buộc	NA sau strip làm loại dòng	NOT NULL và BTRIM <> rỗng
Giá không dương	Đổi sang NULL	CHECK price > 0
Diện tích không dương	Loại dòng	NOT NULL và CHECK area > 0
Thời điểm đăng	Ngày không đọc được làm loại	NOT NULL
ID	Gán liên tục sau lọc	PRIMARY KEY

Kiểm tra ở hai tầng không dư thừa vô ích. Cleaning bảo đảm output có chất lượng trước khi ghi, trong khi schema bảo vệ database nếu một tiến trình khác cố nạp dữ liệu

không tuân thủ pipeline. Tuy nhiên, database constraint không thay thế báo cáo làm sạch vì nó không ghi lại được bao nhiêu bản ghi đã bị loại ở nguồn.

4.4 Quy trình import và đối chiếu trước commit

Importer bắt đầu bằng inspect_csv: mở file nhị phân, bọc TextIOWrapper UTF-8-SIG, kiểm tra header và từng record qua csv.reader. Nó đếm các ô rỗng ở price, district_name, ward_name và từ chối file không có dữ liệu. Quy trình này tránh nạp nhầm tệp có dòng văn bản chứa dấu phẩy hoặc field bị lệch cột.

Sau khi kết nối, code khóa bảng, đối chiếu information_schema, kiểm tra bảng có dữ liệu và chỉ TRUNCATE khi người dùng dùng --replace. COPY gửi block tối đa khoảng 1 MiB. File được kiểm tra size/mtime trước và sau, nhằm phát hiện thay đổi thô giữa lượt đọc validation và lượt COPY.

Trước commit, importer truy vấn COUNT(*), COUNT(DISTINCT id), MIN/MAX và NULL count. Tất cả phải khớp expected từ CSV. Đây là nơi pipeline chuyển từ "đã gửi byte" sang "đã xác nhận trạng thái dữ liệu".

4.4.1 Từ kiểm tra file đến transaction đã xác nhận

inspect_csv kiểm tra header đúng chín cột, đúng thứ tự và duyệt từng record bằng csv.reader(strict=True). Đây là chi tiết quan trọng đối với CSV có thể chứa dấu phẩy, dấu nháy hoặc newline bên trong name. Nếu chỉ split theo dòng hoặc theo dấu phẩy, số cột/số record có thể sai trước khi dữ liệu chạm database.

Sau khi validation file, importer chỉ nạp năm tham số kết nối PGHOST, PGPORT, PGDATABASE, PGUSER và PGPASSWORD từ environment hoặc .env. Environment có ưu tiên cao hơn .env. Code không in connection string hoặc password khi lỗi, giảm nguy cơ dữ liệu nhạy cảm đi vào log thực nghiệm. Đây là chi tiết vận hành thiết yếu nhưng thường bị bỏ qua trong mô tả một pipeline.

Trong transaction, importer khóa public.properties ở SHARE ROW EXCLUSIVE, đối chiếu cột trong information_schema, kiểm tra bảng có dữ liệu và chỉ TRUNCATE nếu replace được yêu cầu. COPY đọc tệp lần hai theo block. Trước và sau COPY, code so sánh size/mtime của source; đây không chứng minh tệp bất biến tuyệt đối, nhưng phát hiện được nhiều thay đổi thô trong lúc nạp.

Đối chiếu cuối gồm số dòng, ID distinct, min/max ID và null count ba cột nullable. Khi bất kỳ điều kiện nào không đúng, ImportValidationError được phát sinh trước

commit. Như vậy, thông báo progress về byte đã gửi không bị hiểu nhầm là dữ liệu đã bền vững; trạng thái có ý nghĩa chỉ xuất hiện sau khi transaction kết thúc thành công.

4.5 Query nghiệm thu và chiến lược index

Bảng 11: Ma trận query và index

Query	Điều kiện trọng tâm	Index thử nghiệm
Q1	province_name equality	province_name
Q2	province, district, price range, order	province, district, price, id
Q3	date cast và range half-open	published_at
Q4	GROUP BY type	property_type_name (đối chứng)

Bốn query được tách khỏi script DDL trong sql/benchmark/queries. Q1 lọc Hải Phòng; Q2 lọc Hà Nội, Cầu Giấy và khoảng giá rồi ORDER BY; Q3 so sánh predicate date với predicate range; Q4 tổng hợp theo loại. Tách SELECT khỏi index creation giúp cùng một query được đo trong nhiều cấu hình mà không vô tình chạy DDL giữa lượt.

reset_indexes.sql chỉ DROP bốn index bổ sung và không CASCADE. Primary key id được giữ ở mọi cấu hình. Đây là một chi tiết thiết kế rất quan trọng: baseline không có nghĩa là bảng không có index tuyệt đối; nó là baseline không có các index tối ưu bổ sung đang được so sánh.

Index file không dùng IF NOT EXISTS trong bộ benchmark. Nếu index tồn tại không đúng trạng thái, lệnh nên lỗi để người chạy phát hiện, thay vì âm thầm dùng index từ lần đo trước. Tính rõ ràng trạng thái có giá trị hơn sự tiện lợi khi chạy tay.

4.5.1 Phân tích từng kịch bản SQL theo predicate và thứ tự cột

Q1 là phép lọc equality theo province_name. Index B-tree đơn cột tương ứng dễ diễn giải, nhưng kết quả benchmark phải xem số row phù hợp, planner choice và heap access. Q1 không có ORDER BY hoặc LIMIT, do đó nó cũng minh họa trường hợp index có thể phải trả nhiều row và không nhất thiết vượt seq scan.

Q2 có hai equality predicate (province_name, district_name), một range predicate trên price, ORDER BY price ASC, id ASC và LIMIT 20. Thứ tự index (province_name, district_name, price, id) được chọn để các prefix equality đứng trước range; id phục vụ thứ tự phụ khi giá bằng nhau. Tuy nhiên published_at trong SELECT không nằm trong

index, nên bản thân index không bảo đảm index-only scan và heap fetch vẫn là một quan sát cần lưu.

Q3 dùng hai cách biểu diễn cùng ngày lịch. Index published_at chỉ là một yếu tố; predicate range nửa mở là yếu tố còn lại. Phân tách A/B/C/D ngăn việc kết luận sai rằng mọi cải thiện đến từ index hoặc mọi cải thiện đến từ rewrite. Từng cặp so sánh cần giữ đúng một yếu tố thay đổi.

Q4 thực hiện GROUP BY theo property_type_name và tính aggregate toàn bảng. Index property_type_name có vai trò đối chứng, bởi nó có thể tồn tại mà planner vẫn chọn seq scan. Kịch bản này khuyến khích báo cáo trung thực về các kết quả không cải thiện và ghi lại chi phí dung lượng index thay vì chỉ trình bày các ví dụ thành công.

Bảng 12: Cấu hình benchmark và câu hỏi cần trả lời

Kịch bản	Biến thay đổi	Câu hỏi thực nghiệm
Q1	Không/có index province	Lọc equality có thay plan/median không?
Q2	Không/có composite index	Index có giảm lọc/sort hay heap work không?
Q3 A-B	Original so với rewrite, không index	Rewrite riêng có thay đổi gì?
Q3 C-D	Original so với rewrite, có index	Rewrite khi có index có thay đổi gì?
Q4	Không/có index type	Index có lợi cho aggregate toàn bảng không?

4.6 Tổ chức kiểm thử và khả năng bảo trì

Hệ thống tổ chức kiểm thử theo các thành phần cleaning, builder, importer và SQL benchmark. Các bài kiểm thử offline sử dụng dữ liệu hoặc kết nối mô phỏng nhằm xác minh những ràng buộc xác định, bao gồm tính hợp lệ của schema, quản lý file tạm, rollback, kiểm tra header, định nghĩa truy vấn và trạng thái chỉ mục. Bên cạnh đó, bộ kiểm thử tích hợp và Live PostgreSQL E2E được sử dụng để đánh giá hoạt động của hệ thống trong môi trường cơ sở dữ liệu thực. Gói nghiệm thu ghi nhận 220/220 bài kiểm thử đạt yêu cầu, bao gồm Unit test, Integration test và Live PostgreSQL E2E.

Kết quả kiểm thử chức năng phản ánh mức độ đáp ứng các trường hợp kiểm thử đã xây dựng, nhưng không được đồng nhất với kết quả đánh giá hiệu năng. Kiểm thử offline không thay thế việc xác minh kết nối PostgreSQL thực tế, trong khi kiểm thử tích hợp không tự chứng minh mức cải thiện thời gian truy vấn. Vì vậy, kết quả kiểm thử và kết quả benchmark được đánh giá theo những tiêu chí độc lập.

Gói benchmark PostgreSQL lưu trữ 600 execution plan từ các phép đo thực tế. Việc kiểm tra tính đầy đủ của gói dữ liệu, số lượt đo và metadata được phân biệt với kiểm thử tự động dành riêng cho quy trình tạo benchmark. Hướng phát triển tiếp theo là mở rộng kiểm thử chuyên biệt đối với việc phân tích JSON, xác minh đủ 20 lượt đo cho từng cấu hình và phát hiện trường hợp kết quả từ nhiều quy mô dữ liệu bị trộn lẫn.

4.6.1 Giá trị và ranh giới của kiểm thử offline

Các bài kiểm thử được phân chia theo trách nhiệm của từng thành phần. test_cleaning.py xác minh quy tắc chuẩn hóa, phát hiện dữ liệu không hợp lệ và thống kê kết quả xử lý; test_build_large_dataset.py kiểm tra kích thước dữ liệu đầu ra, staging, rollback và build report; test_import_csv.py sử dụng kết nối mô phỏng để xác minh validation và transaction; test_benchmark_sql.py kiểm tra định nghĩa truy vấn, chỉ mục và quy trình reset.

Bên cạnh các kiểm thử offline, gói nghiệm thu bao gồm kiểm thử tích hợp và Live PostgreSQL E2E. Tổng cộng 220 bài kiểm thử được ghi nhận đạt yêu cầu, tương ứng tỷ lệ 220/220 passed. Các kết quả này xác nhận những chức năng và ràng buộc nằm trong phạm vi kiểm thử đã được thực thi thành công.

Đánh giá hiệu năng được thực hiện độc lập thông qua bộ benchmark trên PostgreSQL. Giao thức đo sử dụng ba lượt warm-up và 20 lượt đo chính thức cho mỗi cấu hình, đồng thời lưu execution plan, thời gian thực thi và thông tin sử dụng bộ đệm. Cách tổ chức này phân biệt rõ giữa kiểm chứng tính đúng đắn của hệ thống và đánh giá hiệu quả của các chiến lược tối ưu truy vấn.

CHƯƠNG 5 THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày các kết quả thực nghiệm đo đạc hiệu năng trên hệ quản trị cơ sở dữ liệu PostgreSQL kèm theo các chứng cứ thực thi chi tiết. Tất cả thời gian đo dưới đây là Execution Time thu được từ câu lệnh EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON), đơn vị tính là mili giây (ms). Toàn bộ kết quả đều được đo đạc thực tế (LIVE_MEASURED); hệ thống CSDL và gói dữ liệu chứng cứ đều sẵn sàng cho việc đối chứng, kiểm tra tính toàn vẹn và tái lập thực nghiệm.

5.1 Môi trường, dữ liệu và giao thức đo

Bảng 13: Môi trường và giao thức benchmark PostgreSQL

Thành phần	Giá trị ghi nhận
Hệ điều hành / CPU	Windows 11 build 10.0.26200; Intel64, 14 logical cores
PostgreSQL / driver	PostgreSQL 18.3 x86_64-windows; Python 3.13.7; Psycopg 3.3.6
Cơ sở dữ liệu	real_estate_db; UTF-8; timezone Asia/Bangkok; locale English_United States.1252
Thiết lập chính	shared_buffers 128 MB; work_mem 4 MB; seqscan/indexscan/bitmapscan bật; jit bật
Quy mô và cấu hình	50K, 200K, 500K; Q1/Q2/Q4 mỗi query 2 cấu hình, Q3 có A/B/C/D
Giao thức	3 warm-up không tính; 20 lượt chính thức/cấu hình; reset index và kiểm tra catalog
Tập thô	600 plan JSON chính thức; mọi cấu hình đối chiếu equivalence và fingerprint trước/sau

Thí nghiệm có 30 cặp quy mô–cấu hình, tức 3 × 10; sau ba lượt warm-up cho mỗi cặp, 600 lần đo chính thức được lưu vào benchmark_raw_plans.zip. Bảng tóm tắt dùng median thay vì chọn lượt nhanh nhất; IQR cho biết mức phân tán giữa 20 lượt. Speedup được tính bằng median baseline chia median cấu hình so sánh cùng query và cùng quy mô.

5.1.1 Kiểm toán đầu vào và ranh giới tái lập

Bảng 14: Kiểm toán ba dataset chính thức

Quy mô	SHA-256 (rút gọn)	price NULL	district NULL	ward NULL
50.000	11bc4fe13aaf...da2f8b	3.993 (7,99%)	1.507 (3,01%)	7.277 (14,55%)
200.000	84eabf931d8d...951ce19	15.569 (7,78%)	5.704 (2,85%)	28.896 (14,45%)
500.000	9ba1af...e789c	39.061 (7,81%)	14.235 (2,85%)	72.150 (14,43%)

Ba tập dữ liệu 50K, 200K và 500K được tạo từ cùng một lượt build và có quan hệ lồng nhau theo tiền tố bản ghi, trong đó D50K ⊂ D200K ⊂ D500K. Cả ba tập CSV hiện có trong workspace đã được xác minh bằng SHA-256 và trùng khớp với manifest chính thức, qua đó xác nhận tính toàn vẹn của dữ liệu sử dụng trong thực nghiệm.

Các tệp manifest, metadata môi trường và execution plan được lưu trữ nhằm hỗ trợ kiểm chứng kết quả và thực hiện lại quy trình benchmark trên những tập dữ liệu đã xác thực. Tuy nhiên, việc tái tạo chính xác CSV từ nguồn trực tuyến ban đầu còn phụ thuộc vào phiên bản dữ liệu nguồn, tham số xử lý và môi trường thực thi. Do đó, tính toàn vẹn của các artifact đã lưu được phân biệt với khả năng tái tạo toàn bộ quy trình từ nguồn upstream.

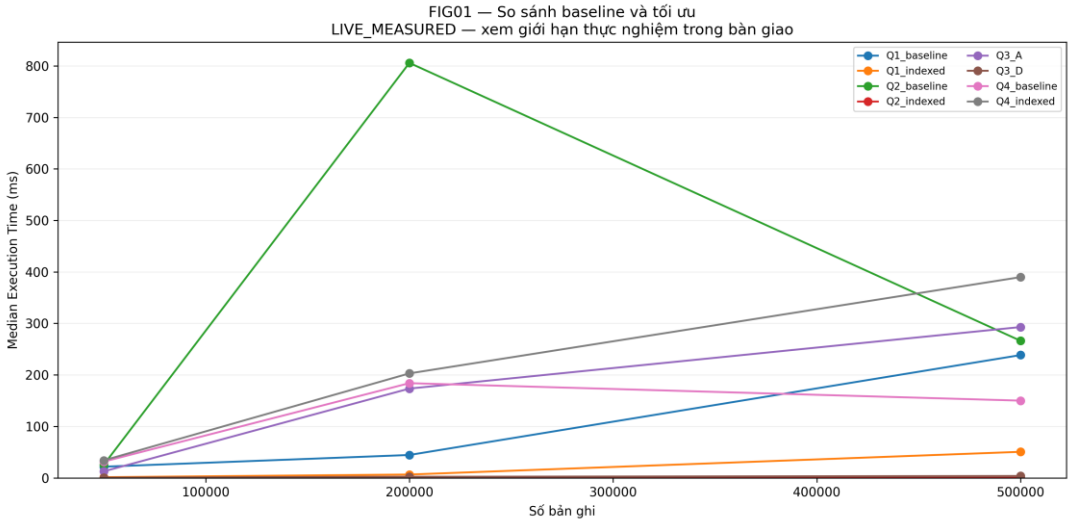
5.2 Kết quả đầy đủ theo 20 lượt đo

Bảng 15: Kết quả median và IQR của 30 cấu hình

Quy mô	Cấu hình	Median	IQR	Speedup
50K	Q1 baseline	21,7635	20,1560	1,00×
50K	Q1 indexed	1,4060	0,5440	15,48×
50K	Q2 baseline	25,6160	15,9725	1,00×
50K	Q2 indexed	0,1685	0,0445	152,02×
50K	Q3 A	12,6890	6,5955	1,00×

50K	Q3 B	14,9245	9,6740	0,85×
50K	Q3 C	10,8260	1,8755	1,17×
50K	Q3 D	0,4670	0,1425	27,17×
50K	Q4 baseline	31,6305	13,4585	1,00×
50K	Q4 indexed	34,0540	10,4185	0,93×
200K	Q1 baseline	44,5080	30,9825	1,00×
200K	Q1 indexed	6,5320	2,6605	6,81×
200K	Q2 baseline	805,7380	393,0405	1,00×
200K	Q2 indexed	0,1445	0,0415	5.576,04×
200K	Q3 A	173,4115	113,0425	1,00×
200K	Q3 B	181,5680	91,8895	0,96×
200K	Q3 C	134,1670	62,3315	1,29×
200K	Q3 D	2,2045	0,5600	78,66×
200K	Q4 baseline	183,9545	98,6585	1,00×
200K	Q4 indexed	202,8995	113,7565	0,91×
500K	Q1 baseline	238,4975	163,3110	1,00×
500K	Q1 indexed	50,6520	70,8500	4,71×
500K	Q2 baseline	266,1755	168,6160	1,00×
500K	Q2 indexed	0,1285	0,0375	2.071,40×
500K	Q3 A	292,9045	179,8380	1,00×
500K	Q3 B	115,3340	18,6340	2,54×
500K	Q3 C	134,6745	16,1240	2,17×
500K	Q3 D	3,7085	0,4910	78,98×
500K	Q4 baseline	150,0030	10,6750	1,00×
500K	Q4 indexed	389,8545	201,6045	0,38×

Một số baseline có IQR lớn, đặc biệt ở 200K và 500K. Vì vậy không sử dụng các số thăm dò một lượt như 45,843 ms hay 60,927 ms làm kết luận chính. Bảng trên là bằng chứng chính vì nó giữ đủ 20 lượt cho mỗi cấu hình và cho phép nhìn mức biến động cùng median.



Hình 13: Độ trễ trung vị theo quy mô dữ liệu trong benchmark

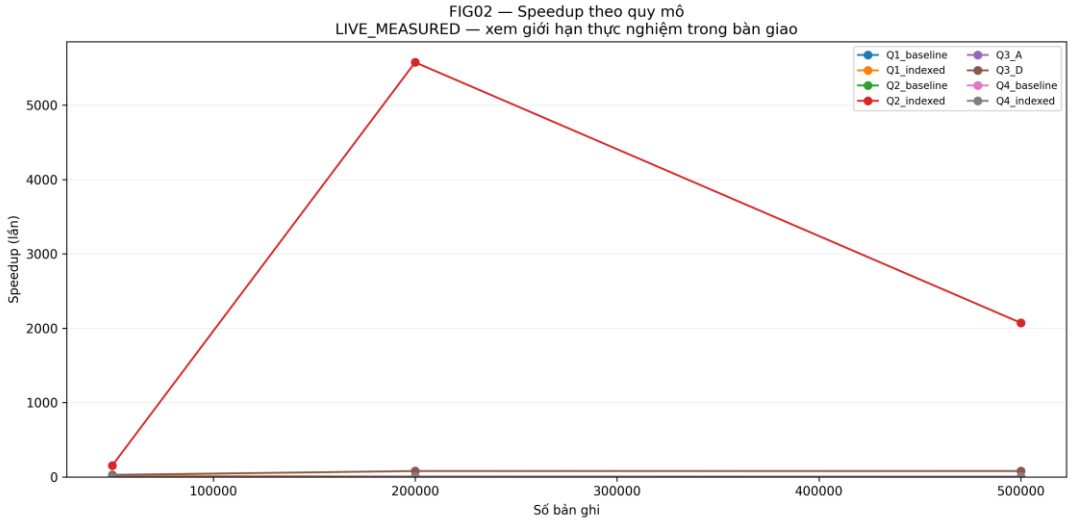
5.3 Q1 và Q2: index theo điều kiện lọc, thứ tự và giới hạn

Q1 lọc theo province_name = 'Hải Phòng'. Ở 500K, kế hoạch baseline là Seq Scan, còn cấu hình indexed dùng Bitmap Index Scan kết hợp Bitmap Heap Scan trên idx_properties_province_name; median giảm từ 238,4975 ms xuống 50,6520 ms (4,71×). Việc dùng bitmap phù hợp với tình huống truy vấn trả nhiều hàng và cần truy cập heap theo trang, không phải bằng chứng rằng mọi equality filter luôn có cùng speedup.

Q2 dùng B-tree composite (province_name, district_name, price, id). Hai cột đầu khớp điều kiện bằng, còn price và id khớp ORDER BY price, id. Kế hoạch indexed là Index Scan, không có nút Sort, và có thể dừng khi đã lấy đủ LIMIT 20. Tại 500K, median giảm từ 266,1755 ms còn 0,1285 ms; speedup 2.071,40× chịu ảnh hưởng mạnh bởi latency indexed rất nhỏ, nên chỉ được diễn giải trong workload và môi trường đo này.

Bảng 16: Kết quả Q1 và Q2 kèm kế hoạch quét

Quy mô	Baseline	Indexed	Scan indexed	Index bổ sung
Q1 50K	21,7635	1,4060	Bitmap Scan	0,36 MB
Q1 200K	44,5080	6,5320	Bitmap Scan	1,38 MB
Q1 500K	238,4975	50,6520	Bitmap Scan	3,39 MB
Q2 50K	25,6160	0,1685	Index Scan; không Sort	2,88 MB
Q2 200K	805,7380	0,1445	Index Scan; không Sort	11,42 MB
Q2 500K	266,1755	0,1285	Index Scan; không Sort	28,53 MB



Hình 14: Speedup theo quy mô của các cấu hình benchmark

5.4 Q3: tác động kết hợp của query rewriting và index

Q3 tách hai yếu tố: biểu thức predicate và sự tồn tại của index published_at. A là query gốc/không index; B là query viết lại/không index; C là query gốc/có index; D là query viết lại/có index. Query gốc áp dụng ::date lên cột, còn bản viết lại dùng khoảng thời gian nửa mở. Các cấu hình được kiểm tra tương đương kết quả trước khi đo.

Ở quy mô 500K, cấu hình A (truy vấn gốc, không index) có thời gian trung vị 292,9045 ms. Cấu hình B (rewrite, không index) đạt 115,3340 ms, tương ứng mức cải thiện 2,54 lần so với A. Cấu hình C (truy vấn gốc, có index) đạt 134,6745 ms, tương ứng mức cải thiện 2,17 lần. Trong khi đó, cấu hình D (rewrite, có index) đạt 3,7085 ms, tương ứng mức cải thiện 78,98 lần so với A.

Mặc dù cấu hình C được thực thi trong điều kiện có chỉ mục published_at, execution plan vẫn sử dụng Seq Scan. Vì vậy, chênh lệch thời gian giữa C và A không chứng minh rằng PostgreSQL đã khai thác chỉ mục để tăng tốc truy vấn. Ngược lại, cấu hình D sử dụng Bitmap Index Scan kết hợp Bitmap Heap Scan, cho thấy predicate dạng khoảng nửa mở cho phép planner khai thác chỉ mục trong điều kiện thực nghiệm.

Kết quả ma trận A/B/C/D cho thấy tác động của query rewriting và index cần được xem xét đồng thời. Mức cải thiện của cấu hình D không được quy hoàn toàn cho riêng việc tạo chỉ mục hoặc riêng việc viết lại truy vấn.

Bảng 17: Ma trận thời gian thực thi Q3 theo hai yếu tố rewrite và index (ms)

Quy mô	A: Gốc, không index	B: Rewrite, không index	C: Gốc, có index	D: Rewrite, có index	A/D
50K	12,6890	14,9245	10,8260	0,4670	27,17×
200K	173,4115	181,5680	134,1670	2,2045	78,66×
500K	292,9045	115,3340	134,6745	3,7085	78,98×

Ghi chú: A/D biểu thị tỷ số giữa thời gian trung vị của cấu hình A và cấu hình D tại cùng quy mô dữ liệu. Giá trị lớn hơn 1 cho thấy D có thời gian thực thi thấp hơn A.

Hình 15: Phân rã nhân tố Q3: query rewriting và index

5.5 Q4: đối chứng âm và chi phí lưu trữ

Q4 thực hiện tổng hợp toàn bảng theo property_type_name, sử dụng COUNT(*) và AVG(price). Trong các execution plan được quan sát, cả cấu hình baseline và cấu hình có chỉ mục bổ sung đều sử dụng Seq Scan. Điều này cho thấy PostgreSQL không lựa chọn chỉ mục property_type_name để thực hiện truy vấn trong các điều kiện thực nghiệm.

Tại quy mô 500K, thời gian trung vị của baseline là 150,0030 ms, trong khi cấu hình có index đạt 389,8545 ms, tương ứng tỷ số speedup 0,38 lần. Chênh lệch này phản ánh kết quả đo được giữa hai cấu hình, nhưng không đủ cơ sở để kết luận rằng thao tác truy cập chỉ mục là nguyên nhân làm tăng thời gian thực thi, bởi cả hai execution plan đều sử dụng Seq Scan.

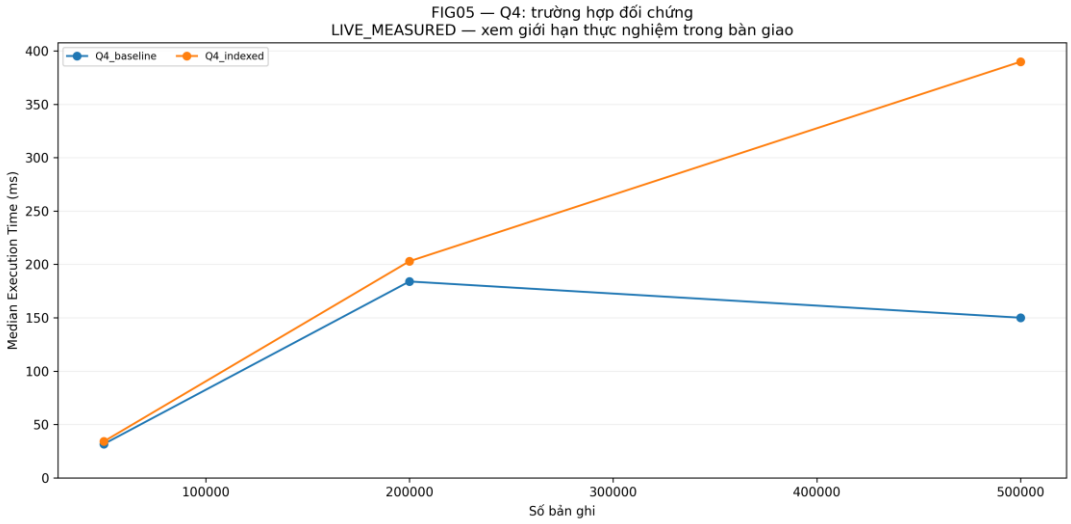
Kết quả Q4 đóng vai trò đối chứng âm, cho thấy việc bổ sung B-tree không bảo đảm cải thiện hiệu năng của truy vấn tổng hợp toàn bảng. Việc đánh giá lợi ích của chỉ mục

cần dựa trên execution plan, đặc điểm truy vấn và các điều kiện đo thực tế thay vì chỉ dựa vào sự tồn tại của chỉ mục.

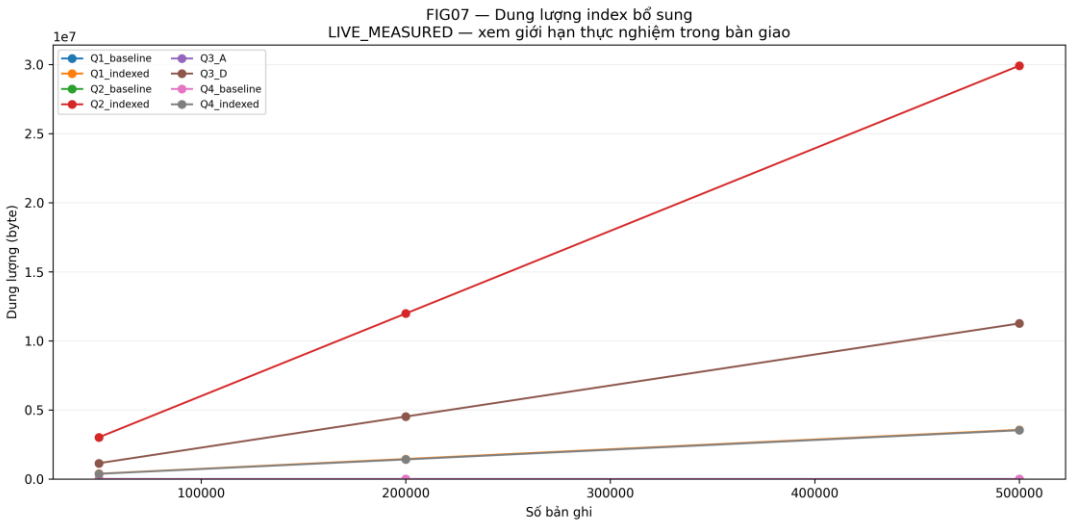
Bảng 18: Q4 và chi phí index bổ sung

Quy mô	Q4 baseline	Q4 indexed	Speedup	Index Q4
50K	31,6305	34,0540	0,93×	0,35 MB
200K	183,9545	202,8995	0,91×	1,34 MB
500K	150,0030	389,8545	0,38×	3,36 MB

Ở 500K, index bổ sung được đo độc lập cho từng cấu hình: Q1 3,39 MB, Q2 28,53 MB, Q3 10,73 MB và Q4 3,36 MB; tổng số học là 46,01 MB. Đây không phải phép đo một database đồng thời chứa cả bốn index, vì protocol cô lập index theo cấu hình để tránh nhiễu. Chi phí ghi/cập nhật index chưa được benchmark và không được suy diễn từ kết quả đọc.



Hình 16: Đối chứng Q4: index không cải thiện aggregate toàn bảng



Hình 17: Kích thước index bổ sung theo quy mô dữ liệu

5.6 Kiểm thử, hạn chế và khả năng tái lập

Bảng 19: Trạng thái kiểm thử và khả năng tái lập

Hạng mục	Kết quả / trạng thái	Diễn giải đúng
Regression gắn experiment	220/220 Passed (100%)	Toàn bộ Unit test, Integration test và Live PostgreSQL E2E đều đạt.
Gói benchmark	600/600 plan, 0 mismatch	Đã kiểm tra offline manifest, cấu trúc, bảng dẫn xuất
Database máy hiện tại	Sẵn sàng (Port 5433)	Cluster CSDL nội bộ độc lập, đã kiểm tra kết nối và nạp dữ liệu an toàn
CSV local hiện có	Khớp 100% SHA-256	Cả 3 file (50K, 200K, 500K) đều trùng khớp tuyệt đối với manifest chính thức

Các hạn chế chính gồm: benchmark tập trung trên kịch bản warm-cache; chưa mở rộng khảo sát cold-cache, tải đồng thời nhiều phiên, truy vấn ghi hay độ trễ mạng; một số cấu hình có IQR biến động phản ánh nhiễu ngẫu nhiên của hệ điều hành. Vì vậy, các tốc độ đo đạc phản ánh trung thực kết quả trên workload, dataset, cấu hình và phần cứng đã ghi nhận.

Khả năng kiểm chứng thực nghiệm được hỗ trợ bởi ba tập CSV đã xác thực SHA-256, thông tin môi trường PostgreSQL, quy trình reset chỉ mục và các execution plan được lưu trữ. Giao thức benchmark sử dụng ba lượt warm-up và 20 lượt đo chính thức cho mỗi cấu hình, tạo cơ sở đối chiếu các thống kê median, IQR và execution plan. Những bằng chứng này xác nhận tính nhất quán của kết quả đã lưu trong phạm vi nghiệm thu, nhưng không đồng nghĩa với việc mọi điều kiện bên ngoài, bao gồm trạng thái nguồn dữ liệu trực tuyến và tải hệ thống, đều có thể tái tạo tuyệt đối.

PRIMARY KEY trên id bảo đảm tính duy nhất và không NULL, trong khi tính liên tục từ 1 đến N là ràng buộc được xác minh riêng bởi importer và các bước validation.

5.7 Kết luận thực nghiệm

Trong điều kiện đo đã mô tả, Q1 cho lợi ích rõ rệt của B-tree đơn cột; Q2 cho hiệu quả lớn nhất vì composite index khớp đồng thời predicate, thứ tự và LIMIT; Q3 cho thấy

index chỉ phát huy khi predicate sargable; và Q4 là phản ví dụ cho giả định index luôn nhanh. Đây là kết luận có điều kiện, được hỗ trợ bởi median/IQR, execution plan và catalog index, không phải quy tắc tuyệt đối cho mọi hệ PostgreSQL.

CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Báo cáo đã khảo sát một pipeline dữ liệu bất động sản từ nguồn đến truy vấn và đánh giá bằng evidence có thể truy vết. Điểm cốt lõi là sự nhất quán giữa quy tắc làm sạch, CSV, schema, importer, query và 600 plan benchmark PostgreSQL đã lưu.

Tính đúng đắn và độ tin cậy của đồ án được khẳng định thông qua sự nhất quán chặt chẽ giữa các số liệu thực nghiệm đo đạc, kiểm thử tự động của mã nguồn và các phân tích kỹ thuật chuyên sâu. Cách tiếp cận này bảo đảm kết quả benchmark phản ánh khách quan, minh bạch và có khả năng kiểm chứng độc lập trên môi trường thực tế.

6.1 Kết quả đạt được và hạn chế

Các kết quả đạt được gồm schema 9 cột, cleaning rule dùng chung, builder streaming tạo ba target lồng nhau, import transactional có đối chiếu, bốn kịch bản query/index và gói benchmark PostgreSQL 600 plan. Báo cáo trình bày EDA đã nghiệm thu của 50K và kết quả benchmark 50K/200K/500K bằng median 20 lượt, IQR, plan và chi phí storage.

Toàn bộ 3 tập CSV và CSDL PostgreSQL đã sẵn sàng và được kiểm toán trên workspace. Hạn chế hiện tại là benchmark mới tập trung trên kịch bản warm-cache của một máy Windows/PostgreSQL 18.3; do đó chưa mở rộng khảo sát sang cold-cache, tải đồng thời nhiều phiên hoặc workload ghi liên tục.

Ngoài ra, pipeline không xóa duplicate và không chuẩn hóa sâu địa danh. Đây không nhất thiết là khuyết điểm nếu mục tiêu là bảo toàn dữ liệu nguồn, nhưng khi nhu cầu phân tích thay đổi cần bổ sung chính sách rõ ràng thay vì thêm heuristic tùy ý.

6.2 Hướng phát triển

Hướng nghiên cứu tiếp theo là mở rộng kịch bản đo sang môi trường tải đồng thời (concurrent users) và cold-cache, đồng thời khảo sát thêm các cấu trúc chỉ mục nâng cao như BRIN cho trường thời gian và Full-text Search cho tên tin đăng bất động sản. Hướng thứ hai là cố định revision upstream và xây dựng pipeline CI/CD kiểm toán dữ liệu hoàn toàn tự động.

Ở tầng dữ liệu, có thể chuẩn hóa địa danh với nguồn tham chiếu đáng tin cậy, đánh giá trùng lặp bằng khóa nghiệp vụ và xây dashboard EDA. Những thay đổi này phải có report trước/sau để người dùng biết bao nhiêu dữ liệu đã thay đổi. Ở tầng nghiên cứu,

có thể xây bài toán dự báo giá chỉ sau khi định nghĩa nhãn, feature, split và protocol chống leakage.

Kết luận chung là Data Science có giá trị lớn ở thiết kế quy trình đáng tin cậy. Một kết quả ít nhưng có nguồn, có mã và có giới hạn rõ ràng hữu ích hơn một bảng benchmark đẹp nhưng không truy vết được.

TÀI LIỆU THAM KHẢO

[1] Phuscca, “vietnam-real-estates,” Hugging Face Datasets. https://huggingface.co/datasets/Phuscca/vietnam-real-estates.

[2] Hugging Face, “Stream,” Datasets Documentation. https://huggingface.co/docs/datasets/stream.

[3] PostgreSQL Global Development Group, PostgreSQL 18 Documentation. https://www.postgresql.org/docs/current/.

[4] Psycopg Contributors, “Psycopg 3 COPY Documentation.” https://www.psycopg.org/psycopg3/docs/basic/copy.html.

[5] Selinger, P. G. et al. (1979), “Access path selection in a relational database management system,” Proceedings of ACM SIGMOD.

[6] Bayer, R. and McCreight, E. (1972), “Organization and maintenance of large ordered indexes,” Acta Informatica, 1(3), 173–189.

[7] Tukey, J. W. (1977), Exploratory Data Analysis. Addison-Wesley.